NAPQES: A Single-Primitive Authenticated Encryption Scheme Built from HMAC-SHA256

Abdeslam Jakjoud[1] and Karim Amor[1]

[1]QAegis, France

Specification revision 4.2 — October 2026
(file `docs/napseq-eprint-v4.tex`; revision history in Section)

Abstract

We present NAPQES, an Authenticated Encryption with Associated Data (AEAD) scheme whose security reduces exclusively to the pseudorandomness of HMAC-SHA256. Unlike AES-GCM and ChaCha20-Poly1305, NAPQES employs no finite-field arithmetic, group operations, or AES S-box permutations, and therefore admits no analogue of the algebraic key-recovery attacks that follow from nonce reuse in a polynomial authenticator. Like every symmetric AEAD scheme, and like the two above, it is unaffected by Shor's algorithm; its post-quantum posture is governed by Grover search alone (Section 5). The construction keys two independently sampled secrets: an arithmetic-layer key $\mathbf{k}$, an ordered tuple of primes driving a prime-indexed token cipher, and a uniform 256-bit secret sk. From sk, a per-wire-format subkey $\mathsf{sk}_{\mathrm{fmt}} = \mathrm{HMAC}(\mathsf{sk}, \texttt{0x0B} \| \mathrm{format_id})$ is derived once (domain `0x0B`; format `0x01` for the block mode specified in this paper, Section 3.3), and every subsequent HMAC domain derivation used for noise injection, plaintext padding, keystream masking, message authentication, and synthetic-nonce derivation is keyed by $\mathsf{sk}_{\mathrm{fmt}}$. This layer is designed to bind every derivation to one wire format so that a tag produced under one format cannot verify under another sharing the same $(\mathbf{k}, \mathsf{sk})$; this paper specifies and proves only the block format, and the separation from the experimental streaming format is not proved (Section 3.3). The nonce is derived synthetically as $N = \mathrm{HMAC}(\mathsf{sk}_{\mathrm{fmt}}, \texttt{0x0A} \| \mathrm{be8}(|A|) \| A \| M)[0{:}16]$, in the manner of AES-GCM-SIV, rather than sampled independently at random. Because $\mathbf{k}$ and sk are sampled independently and the only HMAC calls in the construction are keyed either by sk (the single `0x0B` format-subkey call) or by the sk-derived $\mathsf{sk}_{\mathrm{fmt}}$ (every other domain), we prove IND-CPA-det (the restricted-query notion appropriate to a deterministic, misuse-resistant scheme's synthetic nonce), INT-CTXT, and IND-CCA-det security (Theorems 4.8, 4.18, 4.20) under the standard, uniform-key PRF assumption on HMAC-SHA256 — with no key-entropy term and no non-standard key-distribution assumption — and show that nonce misuse (re-encrypting an identical associated-data/message pair) degrades gracefully to disclosed message-equality leakage rather than key recovery. Ciphertext length is proved to be a deterministic function of the padded-length bucket alone (Lemma 3.13), never of message content, nonce, or associated data. Against a length-observing adversary we define and prove LH-IND-CPA-det (Theorem 4.12), under which the challenge messages may differ in length, at the same bound; we bound the residual length leakage at $\log_2 13 \approx 3.70$ bits per message, and at zero under the frame padding profile (Corollary 4.14); and we exhibit a zero-query adversary breaking the same notion outright for AES-GCM and ChaCha20-Poly1305. We further show that this property derives from the padding profile alone and not from the arithmetic layer, whose removal changes no theorem in the paper (Propositions 4.15 and 6.1). We discuss the post-quantum security level and implementation notes across Python, Rust, and C reference implementations, and report the cross-implementation Known-Answer Test corpora, a NIST SP 800-22 Rev. 1a statistical analysis of ciphertext output, and fuzz coverage of the decoder — evidence for implementation agreement and decoder robustness respectively, not for any security claim, which rests on the theorems alone. A reference implementation, KAT corpus,

and fuzz harnesses are published at `https://github.com/Synergeon-epineon/NAPQES`.

Contents

Revision history **3**

1 Introduction **3**

2 Related Work **4**

3 Construction **5**
3.1 Notation . . . 5
3.2 Key Generation . . . 5
3.3 Domain-Separated HMAC Derivation . . . 7
3.4 Noise Probability . . . 9
3.5 Plaintext Padding . . . 10
3.6 Padding Profiles . . . 10
3.7 Token Construction . . . 11
3.8 Synthetic Nonce . . . 12
3.9 Wire Format . . . 13
3.10 The NAPQES Algorithm Triple . . . 13

4 Security Analysis **16**
4.1 PRF Advantage and Truncation . . . 16
4.2 IND-CPA-det Security . . . 17
4.3 Length Hiding . . . 19
4.4 INT-CTXT: Ciphertext Integrity . . . 22
4.5 IND-CCA-det Security . . . 23

5 Post-Quantum Analysis **24**

6 What the Arithmetic Layer Does and Does Not Buy **25**
6.1 The Construction Without the Arithmetic Layer . . . 26

7 Comparison with AES-GCM, ChaCha20-Poly1305, and Ascon **27**

8 Implementation **28**
8.1 Python Reference Implementation . . . 28
8.2 Rust Implementation . . . 28
8.3 C Implementation . . . 28
8.4 Constant-Time Considerations . . . 28

9 NIST SP 800-22 Statistical Analysis **30**

10 Known-Answer Tests **30**

11 Fuzz Testing **31**

12 Known Caveats **31**

13 Conclusion **33**

Revision history

Each revision lists the changes that affect interoperability first (*normative*), then corrections to the text. A port written against an earlier revision must apply every normative change listed after it.

4.0 (September 2026; fourth audit round). The v8 block format Π as audited in round 4. Prime interval $\mathcal{P}_1$, the primes in $[10^6, 9.9 \times 10^6]$; default $K = 10$.

4.1 (October 2026; commit `f6ced5d`, fifth audit round). *Normative:* prime interval widened to $\mathcal{P}_2$, the primes in $[10^6, 1.5 \times 10^7)$, and default $K = 13$ (Remark 3.3); Dec step (7) gains the $t \geq a$, divisibility and scalar-value guards; the length prefix must be byte-valued (step (8)). *Text:* remediation of the round-4 findings. This revision was filed as `napseq-eprint-v3.tex` and carried no version marker.

4.2 (this revision; round-5 remediation). *Normative for the block format:* none — the bytes of every v8 block ciphertext and every KAT vector of 4.1 are unchanged; vectors `W023` and `W-E02` were added to pin $\mathcal{P}_2$. *Experimental streaming format* (outside this paper, `SPEC.md` §8.2): the sentinel count is masked and the tags bind the full stream header, so streams written under 4.1 do not verify. *Text:* the interval change is disclosed and Remark 3.3 corrected; Dec gains step (0) and step (7) binds k and the reduced addend before its guards; Remark 3.17, the correctness proof, the security preamble and Lemma 4.7 are stated in terms of $\mathsf{sk}_{\mathrm{fmt}}$; the domain table lists the streaming domains `0x0C`–`0x0F`; the file is renamed `napseq-eprint-v4.tex`.

1 Introduction

Modern AEAD standards—AES-GCM [1] and ChaCha20-Poly1305 [2]—have achieved widespread deployment. Neither is threatened by Shor's algorithm, which applies to hidden-subgroup problems rather than to symmetric primitives [11]. Their algebraic structure matters for a different reason: the polynomial authenticators of both (GHASH over GF(2^{128}), Poly1305) turn a single nonce reuse into recovery of the authentication key and hence into arbitrary forgery [12]. That failure mode, not quantum cryptanalysis, is what NAPQES's design avoids.

NAPQES takes a more conservative approach: the *only* cryptographic primitive in the construction is HMAC-SHA256, approved under NIST FIPS 198-1 and FIPS 180-4. The security of the entire scheme therefore reduces to the pseudorandomness of HMAC-SHA256—a well-studied assumption with no known quantum speedup beyond Grover's $O(\sqrt{N})$ search, which reduces the effective security level from 256 bits to approximately 128 bits while remaining far above all recommended thresholds.

Contributions.

1. We fully specify NAPQES (Section 3): an independent HMAC-domain-derivation secret sk and arithmetic-layer key $\mathbf{k}$, sampled independently, together with a synthetic, message-bound nonce $N = \mathrm{HMAC}(\mathsf{sk}_{\mathrm{fmt}}, \texttt{0x0A} \| \mathrm{be8}(|A|) \| A \| M)[0{:}16]$ (Definition 3.12) and a fixed-width, per-bucket-ceiling wire format (Section 3.9) under which ciphertext length depends only on the padded-length bucket.
2. We give a formal AEAD algorithm triple $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ (Definition 3.14) and prove correctness (Proposition 3.18).
3. We prove IND-CPA-det (Definition 4.4, Theorem 4.8), INT-CTXT (Definition 4.17, Theorem 4.18), and IND-CCA-det (Definition 4.19, Theorem 4.20) security under the *standard, uniform-key* PRF assumption on HMAC-SHA256, with no key-entropy term: because sk is

sampled uniformly and independently of $\mathbf{k}$, every reduction below may sample $\mathbf{k}$ on its own to run the arithmetic layer while answering every HMAC call through external PRF oracles — one keyed by sk for the format-subkey derivation, then one keyed by the resulting $\mathsf{sk}_{\mathrm{fmt}}$ for every other domain — with no simulation gap of any kind.

4. We prove ciphertext length is a deterministic function of the padded-length bucket alone (Lemma 3.13) — never of message content within a bucket, the nonce, or the associated data.

5. We define LH-IND-CPA-det (Definition 4.11), the length-hiding strengthening of IND-CPA-det in which the challenge messages need only share a padding bucket, prove that NAPQES satisfies it at the same bound (Theorem 4.12), quantify the residual leakage over an arbitrary padding profile (Corollary 4.14), and exhibit the zero-query adversary that achieves maximal advantage against AES-GCM and ChaCha20-Poly1305 in the same game (Proposition 4.13).

6. We attribute that property to its actual source. The expansion factor is a public injective function of the padding bucket and so contributes no length confidentiality (Proposition 4.15); the arithmetic layer contributes to no theorem in this paper, and we specify the same construction with that layer removed, together with the bounds it preserves and the bandwidth it recovers (Section 6.1, Proposition 6.1).

7. We provide a post-quantum analysis (Section 5) and a comparison with AES-GCM, ChaCha20-Poly1305, and Ascon (Section 7).

8. We report NIST SP 800-22 Rev. 1a statistical test results on ciphertext output (Section 9), the cross-implementation KAT corpora (Section 10), and a fuzz harness covering the full decoder pipeline (Section 11), and state in each case what the evidence does and does not establish.

9. We discuss known caveats (Section 12): the 16-bit plaintext length cap, the fixed ciphertext expansion factor, the padding length-bucket leak, and the scope of what each of sk and $\mathbf{k}$ actually protects.

2 Related Work

AES-GCM [1]. AES-GCM combines AES-CTR for confidentiality with a GHASH polynomial MAC over GF(2^{128}). Under nonce reuse the two keystreams cancel and GHASH degenerates into a solvable polynomial system, so the authentication key H is recoverable and arbitrary forgeries follow: this is the Forbidden Attack [12]. Separately, Iwata, Ohashi and Minematsu [13] showed that GCM's original security proof was flawed for non-96-bit nonces and repaired the bounds; that work concerns the proof, not a nonce-reuse break, and we cite it only for that. NAPQES uses HMAC-SHA256 for authentication, which does not admit the Forbidden Attack, and derives its nonce synthetically so that reuse across distinct (A, M) pairs is not a caller-controlled failure mode.

ChaCha20-Poly1305 [2]. ChaCha20 is an ARX cipher whose security is not reduced to a well-studied hardness assumption; Poly1305 is again a polynomial MAC, subject to the same class of Forbidden Attacks as GHASH when the one-time key is reused. Neither ChaCha20 nor Poly1305 is approved under current FIPS standards.

Ascon [5]. Ascon (NIST LWC winner) is a sponge-based AEAD targeting lightweight environments. Its security is based on the hardness of distinguishing the Ascon permutation from a random permutation, which involves xor-and-rotate operations over 64-bit words. Ascon has since

been standardised by NIST as SP 800-232 [6], so it is an approved algorithm; the comparison in Section 7 reflects that status.

HMAC-based constructions. HKDF [14] and HMAC-DRBG [15] demonstrate that HMAC-SHA256 is a sufficient basis for key derivation and pseudorandom generation. NAPQES extends this philosophy to the full AEAD context.

Deterministic and misuse-resistant AE. Rogaway and Shrimpton [17] formalise deterministic authenticated encryption (DAE) and the corresponding restricted-query security notion, realised by SIV [3] and AES-GCM-SIV [4]. NAPQES adopts the same synthetic-IV philosophy for its nonce. It does *not* inherit those schemes' security notion: what is proved here is the weaker pair of statements of Section 4, as Remark 4.5 sets out.

3 Construction

3.1 Notation

Throughout this paper:

- $\|$ denotes byte concatenation.
- $\mathrm{HMAC}(K, M)$ denotes HMAC-SHA256 keyed with K over message M.
- $\mathrm{be4}(x)$, $\mathrm{be5}(x)$, $\mathrm{be8}(x)$ denote the 4-, 5-, and 8-byte big-endian encodings of the non-negative integer x (each field is fixed-width; x is assumed to fit). In particular be5 requires $x < 2^{40}$; the counters *ct_pos* and *real_idx* it encodes stay far below that, since MAX_PLAINTEXT_CODEPOINTS and the per-bucket token ceiling cap *ct_pos* at $20(2^{16} + 2) = 1{,}310{,}760$.
- $[0{:}n]_{\mathrm{be}k}$ denotes the first n bytes of a 32-byte HMAC-SHA256 digest, interpreted as an unsigned big-endian integer in $[0, 2^k)$ where $k = 8n$.
- $\mathcal{P}$ denotes the set of prime integers in $[10^6, 1.5 \times 10^7)$, a fixed public system parameter with $|\mathcal{P}| = 892{,}206$ (verified by sieve; the interval is half-open, equivalently the inclusive interval $[10^6, 14{,}999{,}999]$, and both endpoints are composite). Implementations MUST use this interval: a different upper bound yields a different $\mathcal{P}$, a different canonical enumeration, and hence different keys. **This interval is new in revision v4.1** (parameter-set identifier $\mathcal{P}_2$; see the revision history and Remark 3.3). Revision v4 and earlier specified $\mathcal{P}_1$, the primes in $[10^6, 9.9 \times 10^6]$; a port implementing $\mathcal{P}_1$ must be updated, and vectors `W023` (an element above 9.9×10^6, $K = 13$) and `W-E02` (the element 15,000,017, rejected) of Section 10 detect a port that has not been. The upper bound was chosen to coincide with the interval from which the reference implementations' key-establishment component (`kem.rs`, `napqes_kem.py`) derives key elements; that component is not specified in this paper.

3.2 Key Generation

A NAPQES key pair is $(\mathbf{k}, \mathsf{sk})$, sampled by $\mathsf{KeyGen}(1^\lambda)$ as follows.

Definition 3.1 (Key generation)**.** *Draw $\mathbf{k} = (k_1, \ldots, k_K)$, an ordered tuple of K distinct primes sampled uniformly, without replacement, from $\mathcal{P}$ via a cryptographically secure DRBG (default $K = 13$);* independently*, draw $\mathsf{sk} \xleftarrow{\$} \{0,1\}^{256}$ uniformly via a CSPRNG. No function relates sk to $\mathbf{k}$. The serialised arithmetic key is*

$$\mathsf{key_bytes}(\mathbf{k}) = \mathrm{be5}(k_1) \| \mathrm{be5}(k_2) \| \cdots \| \mathrm{be5}(k_K).$$

This is a storage and transport encoding only: $\mathbf{k}$ *enters the construction solely through the token map* $c \mapsto c \cdot k_j + a$*, and no step of* Enc *or* Dec *computes* key_bytes *(the legacy v7 format of Section 8 used it as its HMAC key).*

Remark 3.2 (Concrete sampling algorithm)**.** $\mathcal{P}$ *is fixed as a public, canonical ascending enumeration* $\mathcal{P} = \{p_0 < p_1 < \cdots < p_{|\mathcal{P}|-1}\}$ *(e.g. a precomputed sieve table), so "the idx-th prime of* $\mathcal{P}$*" is well-defined and identical across implementations. Index sampling is by rejection:* SampleIndex() *draws a uniform 32-bit unsigned integer* x *from the CSPRNG, sets* $L = \lfloor 2^{32}/|\mathcal{P}| \rfloor \cdot |\mathcal{P}|$*, and rejects and redraws* x *whenever* $x \geq L$*; once* $x < L$*, it returns* $idx = x \bmod |\mathcal{P}|$*, which is then exactly uniform over* $\{0, \ldots, |\mathcal{P}| - 1\}$ *with no modulo bias.* k_1 *is set to* $p_{\text{SampleIndex()}}$*; for* $j = 2, \ldots, K$*,* k_j *is drawn by repeatedly invoking* SampleIndex() *and rejecting (redrawing) any result equal to an already-chosen* $k_1, \ldots, k_{j-1}$*, until a fresh value is obtained. This realises exactly the uniform distribution over ordered tuples of* K *distinct elements of* $\mathcal{P}$ *assumed by the* $H_\infty(\mathbf{k})$ *computation below. Implementations may instead sample uniformly from the integer interval* $[10^6, 14{,}999{,}999]$ *by unbiased rejection, discarding composites and repeats; this is distributionally identical, does not require the canonical enumeration, and is what the reference implementations do.*

The min-entropy of $\mathbf{k}$ as sampled above is

$$H_\infty(\mathbf{k}) = \log_2\left(\frac{|\mathcal{P}|!}{(|\mathcal{P}| - K)!}\right) \approx K \log_2 |\mathcal{P}| \approx 19.7670\, K \text{ bits},$$

so $H_\infty(\mathbf{k}) = 256.9711$ bits at the default $K = 13$ (key-space size $\approx 2.27 \times 10^{77} \approx 2^{256.97}$), i.e. $2^{128.4855}$ against a Grover search. All figures here are computed from the exact $|\mathcal{P}| = 892{,}206$, not from an approximation to it.

Remark 3.3 (Choice of the default parameter set, and what it does not claim)**.** *Revision v4.1 changed the normative interval from* $\mathcal{P}_1$ *(*$[10^6, 9.9 \times 10^6]$*,* $|\mathcal{P}_1| = 579{,}947$*) to* $\mathcal{P}_2$ *(*$[10^6, 1.5 \times 10^7)$*,* $|\mathcal{P}_2| = 892{,}206$*) and the default* K *from* 10 *to* 13*. The motive for the upper bound* 1.5×10^7 *is alignment with the interval the reference key-establishment component already used; it is* not *the smallest interval meeting any target. At* $K = 13$ *an inclusive upper bound of* 14,257,301 *(*$|\mathcal{P}| = 847{,}186$*) already gives* 128 *bits against Grover search, and over* $\mathcal{P}_1$ *raising* K *alone to* 14 *gives* $H_\infty(\mathbf{k}) = 268.04$ *bits (*134.02 *post-Grover). The post-quantum figure for* $\mathbf{k}$ *is reported for completeness only. On v8,* sk *is the only HMAC key and no theorem of this paper depends on* $H_\infty(\mathbf{k})$ *(Remark 3.4); such a figure is meaningful only for the legacy v7 format, where the prime tuple is the whole key and which no theorem of this paper covers. Since* K *affects neither the wire format nor the ciphertext length (Proposition 4.15), the only cost of moving the default from* $K = 10$ *to* $K = 13$ *is stored key material, which grows from* $5K + 32 = 82$ *to* 97 *bytes; widening the interval changes no serialised size. Validation enforces the same interval: every reference implementation rejects a key element outside* $\mathcal{P}$ *and a tuple of more than* 128 *elements. Keys provisioned under* $\mathcal{P}_1$ *lie inside* $\mathcal{P}_2$ *and remain usable; conversely, a port still enforcing* $\mathcal{P}_1$ *rejects* $1 - (579{,}947/892{,}206)^{13} \approx 99.63\%$ *of keys generated under the new default.*

Remark 3.4 (The role of $\mathbf{k}$ and sk are not symmetric)**.** sk *is the* only *secret that ever keys an HMAC call in this construction (Section 3.3);* $\mathbf{k}$ *is used exclusively, directly, and un-keyed by the arithmetic token map* $c \mapsto c \cdot k_j + a$ *(Section 3.7). Consequently, none of the security theorems of Section 4 depend in any way on* $H_\infty(\mathbf{k})$*: the PRF assumption below is stated, and used, against the standard* uniform-*key distribution of* sk *only.* K *therefore carries no security-theorem-driven minimum in this construction (contrast a single-secret design in which the arithmetic key also keys HMAC, which would require a minimum* K *to keep an offline key-guessing term negligible). We state plainly what* K *does* not *do.* K *is never transmitted, appears nowhere in the wire format, and does not influence* $|C|$*, which by Lemma 3.13 is a function of the padding bucket alone; in particular* K *makes no contribution to the length-hiding property of Section 4.3, and no claim of traffic-analysis resistance anywhere in this paper rests on* K*, on* $H_\infty(\mathbf{k})$*, or on the multi-prime structure (Remark 4.16).* $K = 13$ *is an interoperability default, not a security parameter: any*

$K \geq 1$ is permitted by the construction (the reference implementations cap $K \leq 128$ to bound the per-call cost of key validation), and both parties must agree on it out of band exactly as they agree on $\mathbf{k}$ *itself. A receiver configured with a different tuple (including a different K) still verifies the tag, which is keyed by* $\mathsf{sk}_{\mathrm{fmt}}$ *(derived from* sk *alone), and then reaches the structural checks of* Dec *step (7). Each real token produced under an element k but checked against a differing element k' passes the divisibility guard with probability about $1/k'$, that is between $2^{-23.8}$ and $2^{-19.9}$ over $\mathcal{P}$ — small, but not negligible in the cryptographic sense (CVF-V5-24); a message has at least $R \geq 18$ real tokens, and a mismatch on any element that is used makes all of them pass only with a probability that is the product of these terms. A mismatch therefore surfaces as a decryption error, and only with that small probability as a wrong plaintext.*[1] *What the arithmetic layer is retained for instead is analysed in Section 6. Section 12 discusses, conversely, what protecting* $\mathbf{k}$ *actually buys once* sk *is known.*

3.3 Domain-Separated HMAC Derivation

Per-wire-format subkey (domain `0x0B`). Before any per-message derivation, a fixed *format subkey* $\mathsf{sk}_{\mathrm{fmt}}$ is derived from sk under domain `0x0B`:

Definition 3.5 (Format subkey)**.**

$$\mathsf{sk}_{\mathrm{fmt}} \;=\; \mathrm{HMAC}(\mathsf{sk},\ \mathit{0x0B} \,\|\, \mathrm{format_id}) \;\in\; \{0,1\}^{256},$$

where $\mathrm{format_id} \in \{0,1\}^8$ *is a 1-byte constant identifying the wire format. This paper specifies exactly one format,* $\mathrm{format_id} = \mathit{0x01}$ *(block mode). The identifier* $\mathrm{format_id} = \mathit{0x02}$ *is reserved for an experimental streaming-AE format that is outside the scope of this paper: it has no security definition or proof here, the format-subkey separation between* `0x01` *and* `0x02` *is asserted but not proved (the games of Section 4 query the format subkey for* `0x01` *only), and its release-of-unverified-plaintext and per-key chunk-count caveats are recorded in* `docs/CAVEATS.md` *(CAV-001, CAV-005). Its wire format is specified byte for byte in* `SPEC.md` *§8.2, and unlike the block format a stream begins with the byte* `0x02` *(CVF-V5-2). Domain* `0x0B` *takes no nonce and no context beyond the 1-byte identifier; the derivation need only be invoked once per key install, not per message (an implementation may equally recompute it per call, with identical output).*

The purpose of this layer is *cross-format tag separation*: because every subsequent HMAC call is keyed by $\mathsf{sk}_{\mathrm{fmt}}$ rather than sk, a tag or ciphertext produced under one format_id cannot verify under another sharing the same $(\mathbf{k}, \mathsf{sk})$. The ciphertext itself carries no format_id byte on the wire — format selection is an out-of-band API contract — so a ciphertext produced under one format subkey simply fails the tag comparison at the receiver and is indistinguishable from a wrong-key ciphertext. (This describes the block format; an experimental stream starts with its `0x02` byte.)

Per-message derivations. All *other* internal values are computed as

$$\mathrm{Derive}_d(\mathsf{sk}_{\mathrm{fmt}}, N, \mathsf{ctx}) \;=\; \mathrm{HMAC}(\mathsf{sk}_{\mathrm{fmt}},\ d \,\|\, N \,\|\, \mathsf{ctx}) \qquad (d \neq \texttt{0x0B}),$$

where d is a 1-byte domain separator and every variable-width field inside ctx is either length-prefixed (e.g. $\mathrm{be8}(|A|)\|A$ for AAD A) or terminal ($\widetilde{B}$ in domain `0x03`, M in domain `0x0A`), exactly as Lemma 3.7 uses. Domain `0x0A` is the single further exception: it does not take a nonce as input, since it is the derivation that *produces* the nonce (Definition 3.12); its input shape is instead $\texttt{0x0A}\|\mathrm{be8}(|A|)\|A\|M$.

[1] CVF-36: an earlier revision of this remark stated that "the reference implementations emit a warning below $K = 7$". The Rust port emits no such warning; the Python and C ports keep an advisory warning at key generation, labelled as relevant to the legacy v7 schedule only. The threshold was inherited from that earlier single-secret schedule (where $H_\infty(\mathbf{k})$ entered the bound); the sentence has been removed as Remark 3.4 already establishes that K carries no security requirement in the v8 construction.

Definition 3.6 (Tag input)**.** *The authentication tag is always computed over the single quantity*

$$\mathrm{TagIn}(A, \widetilde{B}) \;=\; \mathrm{be8}(|A|) \,\|\, A \,\|\, \widetilde{B},$$

so that the complete domain-`0x03` HMAC input is $\texttt{0x03}\|N\|\mathrm{TagIn}(A, \widetilde{B})$*. Every occurrence of the tag in this paper — Table 1, the wire format (Section 3.9), both* Enc *and* Dec *(Definition 3.14), and the integrity proof (Theorem 4.18) — denotes this one quantity, so encryptor and decryptor cannot diverge in the width of the length prefix.*

Domain	Function	Context	Output
`0x00`	Noise position	be5(ct_pos)	Boolean
`0x01`	Real-token addend	be5($real_idx$)	$[1, k-1]$
`0x02`	Noise probability	(none)	$[0.75, 0.99)$
`0x03`	Auth tag	$\mathrm{TagIn}(aad, \widetilde{B})$ (Def. 3.6)	32 bytes
`0x04`	Noise character	be5(ct_pos)	$[32, 127]$
`0x05`	Noise-token addend	be5(ct_pos)	$[1, k-1]$
`0x06`	Padding codepoint	be4(pad_idx)	$[32, 126]$
`0x07`	Keystream block	be4($block_idx$)	32-byte block
`0x0A`	Synthetic nonce	$\mathrm{be8}(\lvert A\rvert)\|A\|M$	16 bytes
`0x0B`	Format subkey	format_id (1 byte, no nonce)	32 bytes
*Experimental streaming format only (*format_id = *`0x02`, `SPEC.md` §8.2; not covered by any theorem):*			
`0x0C`	Stream chunk tag	$H\|\mathrm{be4}(i)\|\mathrm{be8}(\lvert A\rvert)\|A\|\widetilde{B}_i$	32 bytes
`0x0D`	Stream sentinel tag	$H\|\mathrm{be4}(C)\|\mathrm{be8}(\lvert A\rvert)\|A\|\mathrm{be8}(n)$	32 bytes
`0x0E`	Stream sub-nonce	$N\|\mathrm{be4}(i)$	16 bytes
`0x0F`	Stream count mask	$H\|\mathrm{be4}(C)$	8 bytes

Table 1: Domain-separated HMAC derivation schedule. Domain `0x0B` is keyed by sk and produces the format subkey $\mathsf{sk}_{\mathrm{fmt}}$ (Definition 3.5); every other domain is keyed by $\mathsf{sk}_{\mathrm{fmt}}$, not by sk directly. In the streaming rows (CVF-V5-2) $H = \texttt{0x02}\|\mathrm{be4}(F)\|N$ is the 21-byte stream header, which replaces the bare nonce, i is the 0-based chunk index, C the chunk count and n the real-codepoint count; the streaming format reuses domains `0x00`–`0x02` and `0x04`–`0x07` under the per-chunk sub-nonce. The nonce N is applied uniformly by Derive_d for $d \notin \{\texttt{0x0A}, \texttt{0x0B}\}$ and is therefore omitted from the Context column above for the block-format rows. In rows `0x01` and `0x05`, $k = k_{(real_idx \bmod K)+1}$ is the key element selected by the real-token counter; for `0x05` this holds even though the context is be5(ct_pos), so the noise tokens of a run use the key element of the real token they precede.

Lemma 3.7 (Domain separation)**.** *Write* $\mathsf{enc}(d, N, \mathsf{ctx})$ *for the HMAC-SHA256 input byte string that* $\mathrm{Derive}_d(\cdot, N, \mathsf{ctx})$ *keys HMAC over, laid out as in Table 1 (keyed by* sk *for* $d =$ *`0x0B` and by* $\mathsf{sk}_{\mathrm{fmt}}$ *for every other* d*; the key does not enter the encoding argument itself). Then* enc *is injective on its* entire *argument domain: for any two argument tuples* $(d_1, N_1, \mathsf{ctx}_1) \neq (d_2, N_2, \mathsf{ctx}_2)$ *with each field drawn from its declared range, the two resulting HMAC-SHA256 inputs are distinct. The quantification is over all well-formed argument tuples,* not *merely over those this construction would itself generate; in particular it covers tuples chosen adversarially, as Theorem 4.18 requires. The statement is unconditional — it is a property of the encoding alone, independent of* sk, $\mathsf{sk}_{\mathrm{fmt}}$*, and of any computational assumption.*

Proof. Two inputs with different domain bytes $d_1 \neq d_2$ differ at byte position 0, regardless of any other field, so cross-domain distinctness is unconditional. Fix a domain $d \notin \{\texttt{0x0A}, \texttt{0x0B}\}$: bytes 1..16 carry the nonce, so if $N_1 \neq N_2$ the two inputs already differ there; assume therefore $N_1 = N_2$, which reduces distinctness to injectivity of the context encoding. For domains $\{\texttt{0x00}, \texttt{0x01}, \texttt{0x02}, \texttt{0x04}, \texttt{0x05}, \texttt{0x06}, \texttt{0x07}\}$ the context is a fixed-width positional counter (or

empty), trivially injective; for domain 0x03, $\mathsf{ctx} = \mathrm{TagIn}(A, \widetilde{B}) = \mathrm{be8}(|A|)\|A\|\widetilde{B}$, and because $\mathrm{be8}(|A|)$ fixes $|A|$, the split between A and $\widetilde{B}$ is unambiguous, so distinct $(A, \widetilde{B})$ pairs never collide. Within domain 0x0A, whose input is $\texttt{0x0A}\|\mathrm{be8}(|A|)\|A\|M$ with no nonce field at all, the same length-prefix argument shows $\mathrm{be8}(|A|)\|A\|M$ is injective in (A, M). Domain 0x0B has the shortest input of any tabulated domain — the two bytes $\texttt{0x0B}\|\mathrm{format_id}$ — and hence cannot collide with any other domain's input, which is at least nine bytes long ($\texttt{0x0A}\|\mathrm{be8}(|A|)$ even for $A = \varepsilon$) or seventeen bytes long (all nonce-carrying domains); within 0x0B itself, distinct 1-byte format identifiers give distinct inputs at byte position 1. Every case is a statement about byte layout that holds for arbitrary field values, so enc is injective on the whole domain: any two distinct argument tuples, whatever their provenance, map to distinct HMAC-SHA256 inputs. □

Remark 3.8 (Audit finding CVF-11: undocumented 0x0B format-subkey layer)**.** *Previous revisions of this section stated the schedule as* $\mathrm{Derive}_d(\mathsf{sk}, N, \mathsf{ctx}) = \mathrm{HMAC}(\mathsf{sk}, d\|N\|\mathsf{ctx})$ *without introducing the 0x0B format-subkey layer, and Table 1's caption asserted "every domain is keyed by* sk *only". The reference implementations (napqes.py::_derive_format_subkey, C/napqes.c::derive_format_subkey, rust/src/lib.rs::derive_format_subkey), however, thread* $\mathsf{sk}_{\mathrm{fmt}}$ *through every per-message derivation rather than* sk*, and the shipped KAT corpus (tests/kat/v8_vectors.json) reproduces byte-for-byte only under* $\mathsf{sk} \mapsto \mathsf{sk}_{\mathrm{fmt}}$*: an independent implementation written from the earlier text produces* none *of the twelve positive vectors. The defect was in the specification, not the port; the extra PRF layer is deliberate and buys cross-format tag separation, so this section, Table 1, Lemma 3.7, the four security theorems of Section 4, the security preamble of Section 4, the correctness proof (Proposition 3.18),* Dec *(step (0)), Lemma 4.7, and the Section 12 "Scope of* sk *and* k *compromise" caveat are stated in terms of* $\mathsf{sk}_{\mathrm{fmt}}$ *as it is actually computed (the preamble, the proof,* Dec *step (0) and the lemma were completed in round 5, CVF-V5-24; an earlier version of this remark claimed the restatement was already complete). Each theorem's bound gains one* $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0)$ *term for the single-query 0x0B hop — a quantitatively negligible correction, since the extra HMAC call is against a uniform 256-bit* sk*, but a necessary bookkeeping change so that the object proved matches the object computed.*

Remark 3.9 (Worked example: format subkey and synthetic nonce for KAT vector W001)**.** *So that an independent implementer can localise a divergence in the first sixteen output bytes rather than only as a whole-ciphertext disagreement, we give the two derivations that key everything else, computed from the paper's text alone against KAT vector W001 of tests/kat/v8_vectors.json (empty message, empty AAD,* $\mathsf{sk} = 2f89bc687a7987f421d041bedb1c371f8ef741991f343e57b84f4b0c7875fa8$ format_id $= 0x01$*):*

$$\begin{aligned}\mathsf{sk}_{\mathrm{fmt}} &= \mathrm{HMAC}(\mathsf{sk},\ 0x0B\|0x01)\\ &= c54c0012905b06e2a36e3242d443267ddd39abf3ef7b16d5592ee6e6e319dc6d,\\ N &= \mathrm{HMAC}(\mathsf{sk}_{\mathrm{fmt}},\ 0x0A\|\mathrm{be8}(0))[0{:}16]\\ &= e5bb595068e24fd56305552256b5d3446.\end{aligned}$$

The 16-byte N matches the first sixteen bytes of W001's ciphertext_hex *(Section 3.9: $C = N\|B\|T$). An implementation that keyed the schedule by* sk *directly would produce a different sixteen-byte prefix, letting the mismatch be diagnosed without a byte-by-byte diff of the full ciphertext.*

3.4 Noise Probability

The noise probability is realised entirely with fixed-width integer arithmetic; no floating-point representation is used anywhere in the v8 construction Π specified by this paper.[2] Fix the public

[2]CVF-30: the legacy v7 wire format retained in the reference implementations for backward compatibility with archived ciphertexts uses an IEEE-754 f64 form of the noise threshold. v7 encryptors are `#[deprecated]` per CVF-17; v7 decryptors are kept so archived ciphertexts remain decodable. The v7 f64 path is documented in the rustdoc of `derive_noise_p` in `rust/src/lib.rs`.

integer constants

$$\theta_{\min} = \left\lfloor \tfrac{75 \cdot 2^{64}}{100} \right\rfloor = 13{,}835{,}058{,}055{,}282{,}163{,}712, \qquad \theta_{\max} = \left\lfloor \tfrac{99 \cdot 2^{64}}{100} \right\rfloor = 18{,}262{,}276{,}632{,}972{,}456{,}099,$$

computed exactly in integer arithmetic (CVF-V5-24): the IEEE-754 product 0.99×2^{64} rounds to a value 163 lower and MUST NOT be used, and let $\tau = \mathrm{Derive}_{0x02}(\mathsf{sk}_{\mathrm{fmt}}, N, \varepsilon)[0:8]_{\mathrm{be64}} \in [0, 2^{64})$ be the raw 64-bit unsigned integer HMAC output (no division involved). The per-message noise threshold is the 64-bit integer

$$\theta(N) = \theta_{\min} + \left\lfloor \frac{\tau \cdot (\theta_{\max} - \theta_{\min})}{2^{64}} \right\rfloor,$$

computed with a single 128-bit intermediate product and one integer division, so that $\theta(N)$ is bit-identical across all conforming implementations regardless of platform or language. Every informal reference elsewhere in this paper to a "noise probability $p \in [0.75, 0.99)$" (e.g. Table 1) denotes this integer threshold, i.e. $p = \theta(N)/2^{64}$. The interval is half-open in fact: the largest attainable value is $\theta_{\min} + (\theta_{\max} - \theta_{\min}) - 1$, so $\theta_{\max}$ itself is never reached.

3.5 Plaintext Padding

Let $n = |P|$ (plaintext length in Unicode codepoints). Define

$$B = \max(16,\ 2^{\lceil \log_2(n+1) \rceil}), \qquad P_d = [n \gg 8,\ n \wedge 0\mathrm{xFF}] \| P \| \mathbf{pad},$$

where $\mathbf{pad} = (\mathbf{pad}_0, \ldots, \mathbf{pad}_{B-n-1})$ is a $(B-n)$-codepoint sequence with

$$\mathbf{pad}_i = \left(\mathrm{Derive}_{0x06}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be4}(i))[0{:}4]_{\mathrm{be32}} \bmod 95\right) + 32 \ \in [32, 126].$$

The total padded length is $R = n + 2 + (B - n) = B + 2$ codepoints; we call B the *padding bucket* of the message and R the *real-token count*. Because B depends only on n (never on the codepoint values themselves), two messages of equal codepoint length always share the same padding bucket and the same real-token count.

3.6 Padding Profiles

The map $n \mapsto B$ above is the *bucket* profile, and it is the normative default. Because it is the sole source of the length-hiding property proved in Section 4.3 (Remark 4.16), we specify it as a replaceable parameter rather than as a hard-wired constant. A *padding profile* is any function $B(\cdot)$ from codepoint counts to block sizes satisfying $B(n) > n$ throughout the message space. This paper specifies three:

bucket **(default).** $B(n) = \max(16,\ 2^{\lceil \log_2(n+1) \rceil})$, with 13 reachable values $\{2^4, 2^5, \ldots, 2^{16}\}$.

$\mathsf{coarse}(g)$**.** $B(n) = 2^{4 + g\lceil (e(n)-4)/g \rceil}$ where $e(n) = \max(4, \lceil \log_2(n+1) \rceil)$ and g divides 12: the same ladder thinned by a stride g, with $12/g + 1$ reachable values. $\mathsf{coarse}(1) = \mathsf{bucket}$, and $\mathsf{coarse}(12)$ leaves only $\{2^4, 2^{16}\}$.

$\mathsf{frame}(F)$**.** $B(n) = F$ for a fixed $F \in \{2^4, \ldots, 2^{16}\}$, on the restricted message space $n < F$: every message pads to one common block size, so exactly one value is reachable.

All three profiles take values in the same 13-element set, so the set of legal real-token counts $R = B + 2$ — and hence the set of legal ciphertext lengths — is identical for all of them. A decryptor consequently needs *no* knowledge of the sender's profile: it recovers R from the token count (Section 3.9) and the true length n from the 2-codepoint length prefix, so any conforming decryptor accepts any profile's output unchanged, and any well-formedness check on R remains a check against the same fixed set. The profile is a sender-side parameter that, unlike $\mathbf{k}$ and K,

need not be agreed: it is never transmitted, does not appear in the wire format, and a mismatch is undetectable and harmless to correctness.

Corollary 4.14 is stated over an arbitrary profile: length leakage is at most $\log_2$(number of reachable values) bits — about 3.70 bits under bucket, and *exactly zero* under frame(F), at the cost of every message paying the F-sized ciphertext. Deployments whose threat model is dominated by traffic analysis should select frame(F) for the largest F their bandwidth budget tolerates; this, and not the expansion factor, is the knob that trades bandwidth for length confidentiality (Remark 4.16).

3.7 Token Construction

For each padded codepoint c, define the following primitives, each an explicit reduction of a 32-byte Derive$_d$ output into its target range:

Definition 3.10 (Token-construction primitives)**.** *For a key element k, position counters ct_pos, real_idx:*

$$\begin{aligned}
\texttt{is_noise}(ct_pos) &= \mathbf{1}\big[\,\mathrm{Derive}_{0x00}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be5}(ct_pos))[0{:}8]_{\mathrm{be64}} < \theta(N)\,\big],\\
\texttt{noise_char}(ct_pos) &= (\mathrm{Derive}_{0x04}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be5}(ct_pos))[0{:}4]_{\mathrm{be32}} \bmod 96) + 32,\\
\texttt{noise_addend}(ct_pos, k) &= (\mathrm{Derive}_{0x05}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be5}(ct_pos))[0{:}4]_{\mathrm{be32}} \bmod (k-1)) + 1,\\
\texttt{real_addend}(real_idx, k) &= (\mathrm{Derive}_{0x01}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be5}(real_idx))[0{:}4]_{\mathrm{be32}} \bmod (k-1)) + 1.
\end{aligned}$$

Remark 3.11 (Modulo bias)**.** *Each finite-range reduction above (and the padding-codepoint reduction of Section 3.5) draws a value pseudo-uniformly from $[0, 2^{32})$ and reduces it modulo a small integer m. Writing $2^{32} = am + r$ with $0 \le r < m$, exactly r residues occur with probability $(a+1)/2^{32}$ and the remaining $m-r$ with probability $a/2^{32}$, so the statistical distance from uniform on $\mathbb{Z}_m$ is at most $m/2^{32}$: below 2^{-25} for the padding codepoint ($m = 95$) and the noise character ($m = 96$), and below 2^{-8} for the addends ($m = k - 1 < 2^{24}$).*

We do not *claim that this bias is "dominated by" the PRF advantage of HMAC-SHA256; that comparison is not meaningful, since a small PRF advantage says nothing about the uniformity of a reduced output. The correct statement is that no result in Section 4 requires any of these reduced values to be uniform. Confidentiality rests entirely on the one-time keystream* $\mathrm{ks}(N)$ *that masks the serialised token blob (Section 3.9), and integrity entirely on the tag; the addends, noise characters and padding codepoints are computed* underneath *that mask and are never observed by the adversary in the clear. The one correctness property that does depend on them,* $\gcd(a, k) = 1$*, follows from the range $a \in [1, k-1]$ and holds for every draw regardless of bias. No rejection sampling is therefore required — not because the bias is negligible relative to some other quantity, but because nothing is claimed that it could invalidate.*

The noise threshold $\theta(N)$ (Section 3.4) deserves an explicit correction here: it is not *bias-free. Although it uses exact integer arithmetic and no modulo reduction, the fixed-point scaling $\theta_{\min} + \lfloor \tau(\theta_{\max} - \theta_{\min})/2^{64} \rfloor$ maps 2^{64} equally likely draws of τ onto approximately $0.24 \cdot 2^{64}$ outputs, i.e. about 4.17 preimages per output. Since each output receives either 4 or 5 preimages, individual outputs deviate from uniform on the range by up to +20% in relative terms, and the total variation distance from the uniform distribution on the image of the map is ≈ 0.033. This is harmless — the only property required of $\theta(N)$ is that it land in the public interval $[\theta_{\min}, \theta_{\max})$, which it does by construction, and that* Dec *recompute it identically, which it does — but it should not be described as exempt. The prime-index sampling of Remark 3.2 is genuinely bias-free, by rejection.*

Finally, the boundary of this argument is narrow and worth stating, because it is a property of the composition rather than of the reductions themselves. Any variant that exposes a reduced value before it is masked, that drops the keystream mask, or that reuses a nonce across two messages

makes these biases immediately relevant, and the analysis above would then have to be redone with explicit bias terms. Implementers must not treat "no rejection sampling needed" as a property of the derivations in isolation.

Real and noise tokens are integers $c \cdot k + a$ for codepoint c (or a noise character), key element $k = k_{(real_idx \bmod K)+1}$ of the tuple $(k_1, \ldots, k_K)$ (selected by the real-token counter for noise tokens too, matching Dec step (7) and Listing 1), and addend a from Definition 3.10. Because k is prime and every addend lies in $[1, k-1]$, $\gcd(a, k) = 1$ and hence $\gcd(\text{token}, k) = 1$ for every token, real or noise.

To keep ciphertext length independent of the message-derived nonce's particular noise realisation (Section 3.9), the number of *consecutive* noise tokens emitted before each real token is capped at a fixed constant MAX_NOISE_RUN = 19, and the total token count is padded, with additional filler tokens structurally identical to genuine noise tokens, up to a fixed ceiling of exactly $R \cdot (\text{MAX_NOISE_RUN} + 1)$ tokens per message:

Listing 1: Token emission (encoding)

```
for c in padded:                          # R codepoints total
    noise_run = 0
    while noise_run < MAX_NOISE_RUN and is_noise(ct_pos):
        k = key[real_idx % K]
        emit noise_char(ct_pos) * k + noise_addend(ct_pos, k)
        ct_pos += 1; noise_run += 1
    k = key[real_idx % K]
    emit c * k + real_addend(real_idx, k)
    ct_pos += 1; real_idx += 1

# Ceiling padding: append filler tokens (identical construction to
# genuine noise tokens) until exactly R * (MAX_NOISE_RUN + 1) tokens
# have been emitted in total.
ceiling = R * (MAX_NOISE_RUN + 1)
while len(tokens) < ceiling:
    k = key[real_idx % K]
    emit noise_char(ct_pos) * k + noise_addend(ct_pos, k)
    ct_pos += 1
```

Because the ceiling depends only on R (hence only on the padding bucket B), the total token count $N_{\text{tok}} = R \cdot (\text{MAX_NOISE_RUN} + 1)$ is, by construction, a deterministic function of the padding bucket alone — regardless of sk, N, A, or the specific message (Lemma 3.13).

3.8 Synthetic Nonce

Definition 3.12 (Synthetic nonce)**.** *For* $\mathsf{sk}_{\text{fmt}} \in \{0,1\}^{256}$ *(the format subkey of Definition 3.5),* $A \in \{0,1\}^*$*,* $M \in \mathcal{M}$ *(encoded as UTF-8 bytes),*

$$N(\mathsf{sk}_{\text{fmt}}, A, M) \;=\; \text{HMAC}\big(\mathsf{sk}_{\text{fmt}},\; \texttt{0x0A} \,\|\, \text{be8}(|A|) \,\|\, A \,\|\, M\big)[0{:}16].$$

This is a synthetic IV in the manner of RFC 5297 SIV / AES-GCM-SIV: it is a deterministic function of the key and the input, not sampled independently at random, so re-encrypting an identical (A, M) pair always reproduces the identical nonce (and hence the identical ciphertext) — the standard, disclosed message-equality trade-off of deterministic, misuse-resistant encryption (Definition 4.4).

3.9 Wire Format

Each token is serialised in a fixed-width 8-byte field,

$$B = \mathrm{be8}(\mathrm{token}[0]) \| \mathrm{be8}(\mathrm{token}[1]) \| \cdots \| \mathrm{be8}(\mathrm{token}[N_{\mathrm{tok}} - 1]),$$

so $|B| = 8N_{\mathrm{tok}}$ bytes, with $N_{\mathrm{tok}} = R \cdot (\mathrm{MAX_NOISE_RUN} + 1)$ as fixed in Section 3.7. B is XOR-masked with a keystream derived from domain `0x07`:

$$\widetilde{B} \;=\; B \oplus \mathrm{ks}(N)[0 : |B|], \qquad \mathrm{ks}(N) \;=\; \mathrm{Derive}_{0x07}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be4}(0)) \,\|\, \mathrm{Derive}_{0x07}(\mathsf{sk}_{\mathrm{fmt}}, N, \mathrm{be4}(1)) \,\|\, \cdots,$$

generating only as many 32-byte blocks as needed to cover $|B|$ bytes (truncating the final block). The authentication tag is

$$T = \mathrm{Derive}_{0x03}(\mathsf{sk}_{\mathrm{fmt}},\; N,\; \mathrm{TagIn}(A, \widetilde{B})),$$

a 32-byte HMAC-SHA256 value binding the (possibly empty) associated data A. The ciphertext is

$$C = N \| \widetilde{B} \| T,$$

a 16-byte nonce, the masked token blob, and the 32-byte tag. A decryptor rejects any input with $|C| < 48$ bytes outright (parse-time floor); the true minimum length of a well-formed ciphertext is larger, since every message — including the empty string — pads to at least $B = 16$ codepoints ($R = 18$ real tokens), giving $N_{\mathrm{tok}} = 18 \cdot 20 = 360$, $|B| = 2880$ bytes, and $|C| = 16 + 2880 + 32 = 2928$ bytes at minimum. In general $|C| = 48 + 160\,(B(|M|) + 2)$ bytes, a closed form in the padding bucket alone.

Lemma 3.13 (Ciphertext length is a deterministic function of the padding bucket)**.** *For any* $\mathbf{k}$*,* sk*,* A*,* M*,*

$$|C| \;=\; 16 \;+\; 8\,R\,(\mathrm{MAX_NOISE_RUN} + 1) \;+\; 32, \qquad R = B(|M|) + 2,$$

where $B(\cdot)$ *is the padding-bucket function of Section 3.5. In particular* $|C|$ *depends on* M *only through* $B(|M|)$ *— never through* sk*,* N*,* A*, or the specific codepoints of* M*.*

Proof. Immediate from the construction of Section 3.7: N_{tok} is fixed, by the ceiling-padding step, to exactly $R \cdot (\mathrm{MAX_NOISE_RUN} + 1)$ tokens regardless of the natural noise realisation, and $R = B(|M|) + 2$ depends only on the codepoint *count* of M (Section 3.5). The wire format then adds a constant 16-byte nonce and 32-byte tag, and masking (XOR) does not change $|B|$. □

3.10 The NAPQES Algorithm Triple

Definition 3.14 (NAPQES AEAD algorithm triple)**.** *NAPQES is the triple* $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *over the following typed spaces. The symbol* Π *denotes this scheme, and only this scheme, throughout Section 4.*

- *Key space* $\mathcal{K} \times \{0,1\}^{256}$*:* $\mathcal{K}$ *is the set of ordered* K*-tuples of distinct elements of* $\mathcal{P}$ *(Section 3.2).*
- *AAD space* $\mathcal{A} = \{0,1\}^*$*: the AAD length is encoded as* $\mathrm{be8}(|A|)$ *throughout (Section 3.3), so any* $|A| < 2^{64}$ *bytes is supported unambiguously, with no practical ceiling.*
- *Message space* $\mathcal{M}$*: finite sequences of Unicode* scalar values *— that is, codepoints in* $[\mathtt{U+0000}, \mathtt{U+10FFFF}] \setminus [\mathtt{U+D800}, \mathtt{U+DFFF}]$ *— of length at most* $\mathrm{MAX_PLAINTEXT_CODEPOINTS} = 2^{16} - 1$*, the limit imposed by the 2-codepoint big-endian length prefix written during padding (Section 3.5). The surrogate range is excluded because the synthetic-nonce derivation (Definition 3.12) hashes the UTF-8 encoding of* M*, and UTF-8 is undefined on unpaired surrogates;* $\mathcal{M}$ *is thus exactly the set of strings that admit a UTF-8 encoding.*

- *Ciphertext space* $\mathcal{C} = \{0,1\}^*$*: every output of* Enc *has length* $16+8R(\text{MAX_NOISE_RUN}+1)+32$ *bytes for* $R = B(|M|)+2$ *(Lemma 3.13), that is, in closed form,*

$$|C| \;=\; 48 + 160\big(B(|M|)+2\big) \text{ bytes};$$

the parse-time rejection floor is 48 bytes, but the true minimum length of a well-formed ciphertext is $48 + 160 \cdot 18 = 2928$ *bytes.*

Remark 3.15 (The token field is wide enough for the whole of $\mathcal{M}$)**.** *Every emitted token has the form* $c \cdot k_j + a$ *with* c *a padded codepoint,* $k_j \in \mathcal{P}$ *and* $a \in [1, k_j - 1]$ *(Section 3.7), so* $c \cdot k_j + a \le (c+1)k_j - 1$*. Taking the extreme values admitted by* $\mathcal{M}$ *and* $\mathcal{P}$ *—* $c =$ `0x10FFFF` *and* $k_j = 14{,}999{,}981$*, the largest prime in* $\mathcal{P}$ *— gives a maximum token of* $16{,}711{,}658{,}831{,}871 \approx 2^{43.93}$*. The fixed 8-byte big-endian token field therefore never overflows, with a margin of a factor of over* 1.1×10^6*. This bound is unconditional: it holds for every* $M \in \mathcal{M}$*, every* $\mathbf{k}$*, and every noise realisation, so Lemma 3.13 and the serialisation are well-defined on the entire message space.*

Remark 3.16 (Implementation domains)**.** *The Python and Rust reference implementations realise* $\mathcal{M}$ *as specified: Rust's* `&str` *excludes surrogates by construction, and the Python port rejects them explicitly. (This holds for Rust's v8 entry points; its legacy v7 byte pair* `encrypt_raw`*/*`decrypt_raw` *maps each byte to one codepoint, as the C port does, and is outside* Π*.) The C port (*`C/napqes.c`*) takes a NUL-terminated* `char *` *and maps each* byte *to one codepoint, so it realises* $\mathcal{M}$ *only for codepoints up to U+00FF (it hashes the UTF-8 encoding of those codepoints for the synthetic nonce, so its ciphertexts match the other two ports on that range, including vector* `W016`*); it cannot represent wider codepoints or an embedded NUL. This is an implementation limitation of that port, not a property of the scheme, and it is recorded in Section 12.*

$\mathsf{KeyGen}(1^\lambda) \to (\mathbf{k}, \mathsf{sk})$**.** *As in Definition 3.1.*

$\mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M) \to C$**.** *(0) derive* sk_{fmt} *(Definition 3.5; it may be cached per key install); (1) compute* $N = N(\mathsf{sk}_{\text{fmt}}, A, M)$ *(Definition 3.12); (2) pad* M *to* P_d *(Section 3.5); (3) run the token-emission loop of Section 3.7 over* P_d *using* $(\mathsf{sk}_{\text{fmt}}, N)$ *to obtain the token vector* $(\text{token}[0], \ldots, \text{token}[N_{\text{tok}} - 1])$*; (4) serialise* B*; (5) mask* $\widetilde{B} = B \oplus \text{ks}(N)$*; (6) compute* $T = \text{Derive}_{0x03}(\mathsf{sk}_{\text{fmt}}, N, \text{TagIn}(A, \widetilde{B}))$ *(Definition 3.6); (7) output* $C = N \| \widetilde{B} \| T$*. The public encryption oracle is* $\mathsf{Encrypt}(\mathbf{k}, \mathsf{sk}, A, M) \triangleq \mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M)$ *— a* pure function *of* (A, M)*, since* N *is computed synthetically rather than sampled.*

$\mathsf{Dec}(\mathbf{k}, \mathsf{sk}, A, C) \to M$ **or** $\perp$**.** *(0) derive* sk_{fmt} *(Definition 3.5; it may be cached per key install, CVF-V5-24); (1) if* $|C| < 48$*, output* $\perp$*; (2) parse* $C = N \| \widetilde{B} \| T'$ *with* N *the leading 16 bytes and* T' *the trailing 32 bytes; (3) let* $N_{\text{tok}} = (|C| - 48)/8$ *and, if this is not a non-negative integer or not divisible by* $\text{MAX_NOISE_RUN} + 1$*, output* $\perp$*; otherwise set* $R = N_{\text{tok}}/(\text{MAX_NOISE_RUN} + 1)$*; (4) compute* $T = \text{Derive}_{0x03}(\mathsf{sk}_{\text{fmt}}, N, \text{TagIn}(A, \widetilde{B}))$ *— the same quantity, with the same* be8 *length prefix, that* Enc *step (6) computes (Definition 3.6); (5) if* $T \neq T'$ *(compared in constant time), output* $\perp$*; (6) if* $R \notin \{B + 2 : B \in \{2^4, \ldots, 2^{16}\}\}$*, output* $\perp$ *(Remark 3.17); (7) else unmask* $B = \widetilde{B} \oplus \text{ks}(N)$*, deserialise the* N_{tok} *fixed-width tokens, and invert the token-emission loop using the same* $(\mathsf{sk}_{\text{fmt}}, N)$*-derived noise-position oracle, with* real_idx *counting real tokens from* 0*. For each real token, first bind (CVF-V5-28) the key element* $k = k_{(\text{real_idx} \bmod K)+1}$ *and the* reduced *addend* $a = (\text{Derive}_{0x01}(\mathsf{sk}_{\text{fmt}}, N, \text{be5}(\text{real_idx}))[0{:}4]_{\text{be32}} \bmod (k-1)) + 1 \in [1, k-1]$ *— exactly the prime and addend* Enc *used at that index; then skip at most* MAX_NOISE_RUN *consecutive noise positions and read one real token* t*; if* $t < a$ *or* $(t - a) \not\equiv 0 \pmod{k}$ *output* $\perp$*; otherwise set* $c = (t - a)/k$*, and if* $c >$ `0x10FFFF` *or* c *is a Unicode surrogate output* $\perp$ *(CVF-27: these guards make step (7) total); stop once* R *real tokens have been extracted, discarding any trailing filler; (8) read the 2-codepoint big-endian length prefix* (p_0, p_1)*; if* $p_0 > 255$ *or* $p_1 > 255$ *output* $\perp$ *(CVF-43: canonical, byte-valued prefix), else set* $n = 256p_0 + p_1$ *and, if* $n > R - 2$*, output* $\perp$ *(Remark 3.17); (9) output the* n *codepoints following the prefix.*

Remark 3.17 (The post-authentication structural checks)**.** *Step (3) depends only on $|C|$, a public quantity, and is performed* before *the tag is computed, so a ciphertext of illegal length is rejected without an HMAC evaluation and the rejection reveals nothing the adversary did not already know. Steps (6), (7) and (8) are performed* after *the tag comparison of step (5), so they are never reachable on an input an adversary without* sk *could construct, and they play no part in the INT-CTXT argument. They are specified nonetheless because a malformed but correctly tagged ciphertext — which a key holder can trivially produce — must be rejected rather than drive an out-of-range read, an allocation sized by a field the caller did not choose, or an arithmetic failure. Totality comes from the explicit guards of each step, not from one step implying another (corrected in round 5, CVF-V5-28: an earlier version of this remark said steps (6) and (8) make step (7) total, which is false): step (6) fixes R to a reachable bucket; the guards of step (7) — $t \geq a$, $(t-a) \equiv 0 \pmod{k}$, and a scalar-value range check on c — make every token inversion defined; and step (8) bounds n by the buffer actually recovered. Divisibility by* MAX_NOISE_RUN $+ 1$ *alone does not imply step (6): $R = 20$ passes step (3) and fails step (6). All three reference implementations enforce these checks in this order; vector* `W-N06` *of Section 10 pins step (3),* `W-N07` *step (6),* `W-N09`*,* `W-N10` *and* `W-N12` *the per-token guards of step (7) ($t < a$, non-divisibility, and a recovered surrogate), and* `W-N08` *and* `W-N11` *step (8).*

Proposition 3.18 (Correctness)**.** *For every λ, every $(\mathbf{k}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$, every $A \in \mathcal{A}$, $M \in \mathcal{M}$:*

$$\mathsf{Dec}(\mathbf{k}, \mathsf{sk},\ A,\ \mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M)) = M.$$

Proof. Let $C = N \| \widetilde{B} \| T$ be the output of $\mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M)$. We first check that the two tag inputs coincide, rather than assuming tag agreement.

By Lemma 3.13, $|C| = 16 + 8R(\text{MAX_NOISE_RUN} + 1) + 32$ with $R = B(|M|) + 2 \geq 1$, so $|C| \geq 2928 > 48$ and $N_{\text{tok}} = (|C| - 48)/8 = R(\text{MAX_NOISE_RUN} + 1)$ is a non-negative integer divisible by MAX_NOISE_RUN $+ 1$: steps (1) and (3) of Dec do not reject, and step (3) recovers exactly the R used by Enc. Step (2) parses the leading 16 bytes as N and the trailing 32 as $T' = T$, leaving the middle bytes as exactly the $\widetilde{B}$ that Enc emitted; in particular N is *read from the ciphertext*, not re-derived, so no circularity with the synthetic-nonce derivation arises. Hence Dec step (4) evaluates Derive_{0x03} on $\text{TagIn}(A, \widetilde{B})$ with the same sk_{fmt} (both derive it from the same sk in their step (0)), the same N, the same caller-supplied A, and the same $\widetilde{B}$ as Enc step (6) did (Definition 3.6). The two tag inputs are byte-identical, so $T = T'$ and step (5) does not reject. Since $R = B(|M|) + 2$ with $B(|M|) \in \{2^4, \ldots, 2^{16}\}$, step (6) does not reject either, and since the length prefix written by Enc satisfies $n = |M| \leq B(|M|) = R - 2$, neither does step (8).

Step (7) then unmasks with $\text{ks}(N)$, the same keystream Enc masked with (a function of sk_{fmt} and the recovered N alone), recovering B and hence the token vector. Enc's token-emission loop and Dec's loop-inversion step both compute the noise-position oracle, addends, and padding codepoints as pure, deterministic functions of $(\mathsf{sk}_{\text{fmt}}, N, \text{position})$, so they compute identical intermediate values at every position and Dec reads exactly the real tokens $t = c \cdot k + a$ that Enc wrote, with the same k and reduced addend $a \in [1, k-1]$. The rejection branches of step (7) are therefore never taken (CVF-V5-24): $t \geq a$ and $(t-a) = c \cdot k \equiv 0 \pmod{k}$ hold, and the recovered c is a padded codepoint — a codepoint of $M \in \mathcal{M}$ (a Unicode scalar value), a padding codepoint in $[32, 126]$, or a length-prefix byte in $[0, 255]$ — so it is a scalar value at most `0x10FFFF`. Nor is the byte-valued test of step (8): Enc writes $(p_0, p_1) = (\lfloor |M|/256 \rfloor, |M| \bmod 256)$ with $|M| \leq 65{,}535$, so $p_0, p_1 \leq 255$. Emission and inversion are therefore exact inverses, and stripping the length prefix and padding yields M. □

Remark 3.19 (Valid ciphertexts are not unique per (A, M))**.** *Because* Dec *reads N from the ciphertext instead of re-deriving it from (A, M), the set of C that decrypt to a given M under a given A is larger than $\{\mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M)\}$: anyone holding* sk *may pick an arbitrary $N' \neq N$,*

rebuild $\widetilde{B}'$ and T' under it, and obtain a second, equally valid ciphertext for the same plaintext. Enc *is a function, but* Dec *is not injective on valid ciphertexts, so NAPQES is* not *tidy in the sense of Namprempre–Rogaway–Shrimpton, and ciphertext equality must not be used as a proxy for plaintext equality by any protocol built on it.*

This costs nothing in the security analysis. INT-CTXT (Definition 4.17) measures freshness over the submitted pair (c, A), not over the underlying plaintext, so an alternative encoding of an already-queried message is a legitimate forgery attempt and is counted as one; Theorem 4.18 bounds it. Re-deriving N inside Dec *and rejecting on mismatch would restore uniqueness, but it would also make* Dec *depend on the plaintext it is trying to recover, and it is not done.*

Remark 3.20 (Authentication-failure output $\perp$)**.** *The formal $\perp$ output of* Dec *is realised in all three reference implementations as an error rather than a sentinel plaintext (`ValueError` in Python, `Err` in Rust, a `NULL` return in C) — a standard, semantically equivalent realisation of $\perp$ (cf. most software AEAD APIs, e.g. `cryptography.hazmat`).*

4 Security Analysis

Every theorem in this section is unconditional under the standard, uniform-key PRF assumption on HMAC-SHA256 stated below: because sk is sampled uniformly and independently of $\mathbf{k}$, every reduction constructed in this section may sample $\mathbf{k}$ on its own (to run the arithmetic layer) while forwarding every HMAC call to external oracles: the single domain-`0x0B` call to one keyed by the challenger's sk, and every per-message call to one keyed by $\mathsf{sk}_{\mathrm{fmt}} = \mathrm{HMAC}(\mathsf{sk}, \texttt{0x0B}\|\texttt{0x01})$, which the first PRF hop of every proof replaces by a uniform 256-bit key (CVF-V5-24). The reduction never needs to extract sk or $\mathsf{sk}_{\mathrm{fmt}}$ from $\mathbf{k}$, or vice versa, so no simulation gap and no non-standard key-distribution assumption arises. NAPQES does not define a standalone MAC verification key separate from $\mathsf{sk}_{\mathrm{fmt}}$, so EUF-CMA unforgeability of the tag and INT-CTXT unforgeability of the ciphertext coincide here.

4.1 PRF Advantage and Truncation

Definition 4.1 (HMAC-SHA256 PRF advantage)**.** *Let $F = \text{HMAC-SHA256} : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}^{256}$. For a PPT distinguisher $\mathcal{B}$ with oracle access to either $F(\kappa, \cdot)$ for $\kappa \xleftarrow{\$} \{0,1\}^{256}$, or a function $\rho : \{0,1\}^* \to \{0,1\}^{256}$ sampled uniformly from all functions of that type,*

$$\mathrm{Adv}_F^{\mathrm{PRF}}(\mathcal{B}) \;=\; \Big|\Pr[\mathcal{B}^{F(\kappa,\cdot)} = 1] - \Pr[\mathcal{B}^{\rho(\cdot)} = 1]\Big|.$$

Assumption 4.2 (Uniform-key HMAC-SHA256 PRF)**.** $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B})$ *is negligible in the security parameter for every PPT $\mathcal{B}$, for κ drawn uniformly from $\{0,1\}^{256}$ (matching how* sk *is sampled, Definition 3.1).*

Lemma 4.3 (Truncation preserves the PRF property)**.** *Let $F : \{0,1\}^{256} \times \{0,1\}^* \to \{0,1\}^{256}$ be a keyed function with 32-byte output and fix a truncation length ℓ in* bytes*, $1 \le \ell \le 32$; write $\mathrm{trunc}_\ell(x) = x[0{:}\ell]$, so that $\mathrm{trunc}_\ell \circ F$ has output length 8ℓ bits. Then for every adversary $\mathcal{B}$ against $\mathrm{trunc}_\ell \circ F$ there is an adversary $\mathcal{B}'$ against F, running in essentially the same time and making the same number of queries, with*

$$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{trunc}_\ell \circ F}(\mathcal{B}) \;=\; \mathrm{Adv}^{\mathrm{PRF}}_F(\mathcal{B}').$$

Proof. Define $\mathcal{B}'$ to be the adversary that runs $\mathcal{B}$, forwards each of $\mathcal{B}$'s queries to its own oracle, truncates each 32-byte answer to its first ℓ bytes before handing it back to $\mathcal{B}$, and finally outputs whatever $\mathcal{B}$ outputs. $\mathcal{B}'$ makes exactly as many queries as $\mathcal{B}$ and adds only the cost of the truncations.

If $\mathcal{B}'$'s oracle is $F(\kappa,\cdot)$ then the view it presents to $\mathcal{B}$ is exactly $(\mathrm{trunc}_\ell \circ F)(\kappa,\cdot)$. If $\mathcal{B}'$'s oracle is a uniformly random ρ with 32-byte outputs, then $\mathrm{trunc}_\ell \circ \rho$ is a uniformly random function with ℓ-byte outputs — each answer is a prefix of an independent uniform 32-byte string, hence uniform on $\{0,1\}^{8\ell}$, and answers at distinct inputs remain independent — which is exactly the ideal object in $\mathcal{B}$'s game. Both probabilities therefore transfer verbatim, and the two advantages are equal. Note the direction of the reduction: it is $\mathcal{B}$ that is simulated by $\mathcal{B}'$, so the equality is between the advantage of $\mathcal{B}$ in one game and that of the *derived* adversary $\mathcal{B}'$ in the other, not between the advantages of a single adversary in two different games. □

This covers every truncated derivation used by this construction (nonce $\ell = 16$, addend and padding codepoint $\ell = 4$, noise threshold $\ell = 8$) under Assumption 4.2 alone. The symbol ℓ is local to this lemma; it is unrelated to the noise-threshold draw τ appearing in the definition of $\theta(N)$ in Section 3.4.

4.2 IND-CPA-det Security

Because $\mathsf{Encrypt}(\mathbf{k},\mathsf{sk},A,M)$ is a pure function of (A,M) (Section 3.10), the standard, unrestricted left-or-right IND-CPA game is not achievable by any deterministic scheme: an adversary who queries the encryption oracle on one of its own two challenge messages trivially learns the challenge bit by comparing the result to the challenge ciphertext. That this game is unachievable is not a defect of the construction but a structural property of determinism, shared by every deterministic scheme including the synthetic-IV designs of Rogaway and Shrimpton [17]. The notion we prove in its place is the restricted-query game below. Remark 4.5 states how that notion relates to, and where it falls short of, the deterministic authenticated-encryption (DAE) notion of [17].

Definition 4.4 (IND-CPA-det security)**.** *For a PPT adversary $\mathcal{A} = (\mathcal{A}_1,\mathcal{A}_2)$, define $\mathbf{Exp}^{\text{ind-cpa-det}}_{\mathcal{A}}(\lambda)$:*

1. $(\mathbf{k},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$.
2. *$\mathcal{A}_1$ is given adaptive oracle access to $\mathsf{Encrypt}(\mathbf{k},\mathsf{sk},\cdot,\cdot)$ and, after at most q total queries (counted jointly with step 4), outputs $(A^*,m_0,m_1,\mathsf{st})$ with $|m_0| = |m_1|$ (codepoint count).*
3. *The challenger samples $b \xleftarrow{\$} \{0,1\}$ and returns $c^* \leftarrow \mathsf{Encrypt}(\mathbf{k},\mathsf{sk},A^*,m_b)$.*
4. *$\mathcal{A}_2(\mathsf{st},c^*)$, with continued oracle access, outputs a bit b'.*
5. *The experiment outputs 1 iff $b' = b$.*

$\mathcal{A}$ must additionally respect two restrictions, or the experiment aborts with $\mathcal{A}$ losing: (i) no repeated queries *— $\mathcal{A}$ never queries the encryption oracle twice on an identical (A,M) pair, in either phase; (ii)* no challenge-message queries *— writing (A^*,m_0,m_1) for $\mathcal{A}_1$'s output, $\mathcal{A}$ never queries the encryption oracle, in either phase, on (A^*,m_0) or (A^*,m_1). Restriction (ii) is the one forced by determinism, as argued above. Restriction (i) costs no generality: because* Encrypt *is a pure function, an adversary that repeats a query already holds the answer, so any $\mathcal{A}$ violating (i) is converted into one respecting it, with identical view and identical advantage, by caching its own query/response pairs and answering repeats locally. The advantage of $\mathcal{A}$ is* $\mathrm{Adv}^{\text{IND-CPA-det}}_{\Pi}(\mathcal{A}) = |\Pr[\mathbf{Exp}^{\text{ind-cpa-det}}_{\mathcal{A}}(\lambda) = 1] - \frac{1}{2}|$.

Remark 4.5 (Scope: this is not a DAE claim)**.** *Definition 4.4 is a* single-challenge, left-or-right *notion. The DAE notion of [17], which AES-GCM-SIV[4] and SIV [3] target, is stronger in two respects: it asks that* every *ciphertext returned over the whole interaction be indistinguishable from uniformly random bits of the same length, not that one designated challenge hide one bit; and it bundles that requirement with ciphertext integrity into a single game. We prove neither of those here. What Theorem 4.8 and Theorem 4.18 establish is the pair of separate, weaker statements, composed into Theorem 4.20 in the usual way. Accordingly, references in this paper to AES-GCM-SIV and to "misuse resistance" denote a shared design philosophy — a nonce*

synthesised from the message so that nonce reuse is not a caller-controlled failure mode — and must not be read as a claim that NAPQES has been proved DAE-secure. We expect the stronger indistinguishability-from-random-bits statement to be reachable for this construction at the cost of the same $q^2/2^{128}$ nonce-collision term the bounds below already carry, by a union bound over queries in place of the single-challenge hybrid; establishing it is left to future work.

Lemma 4.6 (Equal padded length is derived)**.** *$|m_0| = |m_1|$ forces m_0, m_1 to share the same padding bucket B and the same real-token count R (Section 3.5), with certainty — this is a direct consequence of B depending only on codepoint count.*

Lemma 4.7 (Nonce collision)**.** *For any two distinct queried pairs $(A_1, M_1) \neq (A_2, M_2)$ under the same sk_{fmt}, where sk_{fmt} is uniform in $\{0,1\}^{256}$ (as it is in game G_1, after the domain-`0x0B` hop; CVF-V5-24), there is a PRF adversary $\mathcal{B}$ with similar running time such that $\Pr[N(\mathsf{sk}_{\text{fmt}}, A_1, M_1) = N(\mathsf{sk}_{\text{fmt}}, A_2, M_2)] \leq \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}) + 2^{-128}$.*

Proof. By Lemma 3.7, $(A_1, M_1) \neq (A_2, M_2)$ implies the two domain `0x0A` inputs are distinct, so replacing $\mathrm{HMAC}(\mathsf{sk}_{\text{fmt}}, \cdot)$ by a truly random function ρ (a PRF hop of advantage $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B})$) makes the two 128-bit truncated outputs independent and uniform, colliding with probability exactly 2^{-128}. □

Theorem 4.8 (IND-CPA-det)**.** *For any PPT adversary $\mathcal{A}$ respecting Definition 4.4's restrictions and making at most q encryption queries, there exist PRF adversaries $\mathcal{B}_0, \mathcal{B}_1$ with similar running time such that*

$$\mathrm{Adv}^{\text{IND-CPA-det}}_{\Pi}(\mathcal{A}) \;\leq\; \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0) + \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}},$$

against the standard, uniform-key HMAC-SHA256 PRF assumption, with no key-guessing or key-entropy term. The $\mathcal{B}_0$ term accounts for the one-query PRF hop that replaces $\mathsf{sk}_{\text{fmt}} = \mathrm{HMAC}(\mathsf{sk}, \texttt{0x0B}\|\text{format_id})$ (Definition 3.5) by a uniform 256-bit string; the $\mathcal{B}_1$ term is the subsequent PRF hop against $\mathrm{HMAC}(\mathsf{sk}_{\text{fmt}}, \cdot)$, as in the proof below.

Proof. We proceed via a sequence of three hybrid games.

Game G_0 (real). The challenger samples $(\mathbf{k}, \mathsf{sk})$, computes $\mathsf{sk}_{\text{fmt}} = \mathrm{HMAC}(\mathsf{sk}, \texttt{0x0B}\|\text{format_id})$ once (Definition 3.5), and answers queries via the real Enc, which calls $F(\mathsf{sk}_{\text{fmt}}, \cdot)$ for all per-message derivations.

Game G_0' (format-subkey PRF hop). Identical to G_0, except sk_{fmt} is sampled uniformly from $\{0,1\}^{256}$ rather than derived by $\mathrm{HMAC}(\mathsf{sk}, \texttt{0x0B}\|\text{format_id})$. Because sk is uniform on $\{0,1\}^{256}$ (Definition 3.1) and $\texttt{0x0B}\|\text{format_id}$ is a single fixed input, replacing $\mathrm{HMAC}(\mathsf{sk}, \cdot)$ by a truly random function on this one query is a PRF hop of advantage at most $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0)$ for some PRF adversary $\mathcal{B}_0$ running in time $\mathcal{O}(T_{\mathcal{A}})$; the output of that single call is distributed identically to a uniformly sampled 256-bit string, so $|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_0']| \leq \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0)$.

Game G_1 (per-message PRF replacement). Identical to G_0', except $F(\mathsf{sk}_{\text{fmt}}, \cdot)$ is replaced by a truly random function $\rho : \{0,1\}^* \to \{0,1\}^{256}$, sampled lazily.

Lemma 4.9. *There is a PRF adversary $\mathcal{B}_1$, running in time $\mathcal{O}(T_{\mathcal{A}})$, such that $|\Pr[\mathcal{A} \text{ wins } G_0'] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$.*

Proof. $\mathcal{B}_1$ is given an oracle $\mathcal{O}$ that is either $F(\mathsf{sk}_{\text{fmt}}, \cdot)$ (for a uniform 256-bit sk_{fmt}) or $\rho(\cdot)$. Because sk_{fmt} is uniform on $\{0,1\}^{256}$ in G_0' and is sampled independently of $\mathbf{k}$ (Definition 3.1 plus the format-subkey hop above), $\mathcal{B}_1$ samples its own $\mathbf{k}'$ via the prime-sampling step of KeyGen to run the arithmetic layer of Enc directly, and forwards every per-message domain-derivation

call (including domain 0x0A) to $\mathcal{O}$. $\mathcal{B}_1$ never needs to know sk_{fmt} (or the challenger's hidden key) to run this simulation: when $\mathcal{O} = F(\mathsf{sk}_{\text{fmt}}, \cdot)$ the simulation is distributed identically to G'_0; when $\mathcal{O} = \rho$, identically to G_1. □

Nonce collision under G_1. Let Coll be the event that the challenge nonce N^* collides with some queried nonce $N_1, \ldots, N_q$. By Definition 4.4's restriction (ii), every queried pair (A_i, M_i) is distinct from the challenge pair (A^*, m_b) regardless of b, so Lemma 4.7 applies validly to every pair $\{(A_i, M_i), (A^*, m_b)\}$; a union bound over the q queries (together with pairwise collisions among $N_1, \ldots, N_q$) gives $\Pr[\text{Coll}] \le q^2/2^{128}$ (the $\text{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$ term from Lemma 4.7 is already accounted for by $\mathcal{B}_1$ above).

Lemma 4.10 (Hiding)**.** *In G_1, conditioned on $\overline{\text{Coll}}$,* $\Pr[\mathcal{A}$ *wins* $G_1 \mid \overline{\text{Coll}}] = \frac{1}{2}$.

Proof. Under $\overline{\text{Coll}}$, $N^* \notin \{N_1, \ldots, N_q\}$. Every challenge-phase derivation (domains 0x00–0x07) uses an input prefixed by its domain byte and then N^*; by Lemma 3.7 none of these inputs collides with any query-phase input (different nonce at the fixed offset) or with any other domain's input (different domain byte). Since ρ is truly random, all challenge-phase outputs are therefore fresh, uniform, and independent of the adversary's view. In particular the keystream $\text{ks}(N^*)$ (domain 0x07) is independent of the token blob B it masks (which is itself assembled entirely from domains 0x00, 0x01, 0x02, 0x04, 0x05, 0x06), so the one-time-pad identity applies: for fixed B, $\widetilde{B} = B \oplus \text{ks}(N^*)$ is uniform over $\{0,1\}^{8|B|}$. By Lemma 4.6, m_0, m_1 share the same padding bucket and real-token count R; by Lemma 3.13, $N_{\text{tok}} = R(\text{MAX_NOISE_RUN} + 1)$ is therefore identical for m_0 and m_1 (indeed for *any* nonce), so $|B|$ — and hence the distribution of $\widetilde{B}$ — is identical regardless of b. The tag $T = \rho(\texttt{0x03} \| N^* \| \cdots)$ is a fresh uniform 32-byte value, also independent of b. Hence $c^* = (N^*, \widetilde{B}, T)$ is jointly uniform over its domain, independently of b, giving the adversary no information about b. □

Combining: $\Pr[\mathcal{A} \text{ wins } G_1] \le \Pr[\text{Coll}] + \frac{1}{2} \le q^2/2^{128} + \frac{1}{2}$, so

$$\text{Adv}^{\text{IND-CPA-det}}_{\Pi}(\mathcal{A}) = \left|\Pr[\mathcal{A} \text{ wins } G_0] - \tfrac{1}{2}\right| \le \text{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0) + \text{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}},$$

□

where the $\mathcal{B}_0$ term is the $G_0 \to G'_0$ hop above and the $\mathcal{B}_1$ term is the $G'_0 \to G_1$ hop of Lemma 4.9.

4.3 Length Hiding

Definition 4.4 quantifies only over challenge pairs of *equal* codepoint length, and therefore says nothing whatsoever about what an observer learns from $|C|$ itself. The notion below is the one under which a length-observing adversary must be analysed. It is strictly stronger than Definition 4.4 — the challenge messages may differ in length — and it is the only sense in which this paper claims resistance to traffic analysis. Every such claim elsewhere in this paper refers to Theorem 4.12 and Corollary 4.14, and to nothing else.

Definition 4.11 (LH-IND-CPA-det security)**.** *Fix a padding profile $B(\cdot)$ (Section 3.6). The experiment $\mathbf{Exp}^{\text{lh-ind-cpa-det}}_{\mathcal{A}}(\lambda)$ is defined exactly as $\mathbf{Exp}^{\text{ind-cpa-det}}_{\mathcal{A}}(\lambda)$ of Definition 4.4, retaining both restrictions (i) and (ii), except that step 2's condition $|m_0| = |m_1|$ is replaced by the strictly weaker condition*

$$B(|m_0|) \;=\; B(|m_1|);$$

that is, $\mathcal{A}$ may choose challenge messages of different *codepoint lengths, subject only to their falling in the same padding bucket. The advantage $\text{Adv}^{\text{LH-IND-CPA-det}}_{\Pi}(\mathcal{A})$ is defined as before.*

Theorem 4.12 (LH-IND-CPA-det)**.** *For any PPT adversary $\mathcal{A}$ respecting Definition 4.11's restrictions and making at most q encryption queries, there exist PRF adversaries $\mathcal{B}_0, \mathcal{B}_1$ with*

similar running time such that

$$\mathrm{Adv}^{\text{LH-IND-CPA-det}}_{\Pi}(\mathcal{A}) \;\le\; \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0) + \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}},$$

i.e. exactly the bound of Theorem 4.8, with no additional term for the relaxed length condition. The $\mathcal{B}_0$ hop is the same single-query `0x0B` format-subkey hop as in Theorem 4.8.

Proof. The proof of Theorem 4.8 applies verbatim. Inspecting it, the hypothesis $|m_0| = |m_1|$ is invoked in exactly one place: the appeal to Lemma 4.6 inside the proof of Lemma 4.10, whose sole function is to establish that m_0 and m_1 share a padding bucket B, and hence a real-token count $R = B + 2$. Definition 4.11 supplies that conclusion directly as a hypothesis, so Lemma 4.6 is simply not needed. Every other step — the PRF hop (Lemma 4.9), the nonce-collision bound (Lemma 4.7), and the one-time-pad argument — is independent of the challenge messages' lengths. By Lemma 3.13, $|C|$ is then identical for m_0 and m_1, so the ciphertext length carries no information about b, and $\widetilde{B}$ is uniform over strings of that one common length. □

The notion is not vacuous, and it is not satisfied by the schemes NAPQES is compared against in Section 7.

Proposition 4.13 (Separation from AES-GCM and ChaCha20-Poly1305)**.** *Let Σ be any AEAD scheme whose ciphertext length is a strictly increasing function of the plaintext length — in particular AES-GCM and ChaCha20-Poly1305, for which $|C| = |M| + 16$ bytes. Then there is an adversary $\mathcal{A}_{\text{len}}$ making* zero *oracle queries and performing a single integer comparison such that*

$$\mathrm{Adv}^{\text{LH-IND-CPA-det}}_{\Sigma}(\mathcal{A}_{\text{len}}) \;=\; \tfrac{1}{2},$$

the maximum possible advantage, irrespective of the strength of the underlying primitive.

Proof. $\mathcal{A}_{\text{len}}$ outputs any (A^*, m_0, m_1) with $B(|m_0|) = B(|m_1|)$ but $|m_0| \neq |m_1|$ — for instance $|m_0| = 1$ and $|m_1| = 8$, both in bucket $B = 16$ under the default profile. On receiving c^* it returns $b' = 0$ if $|c^*| = |m_0| + 16$ and $b' = 1$ otherwise, winning with probability 1. Restrictions (i) and (ii) are satisfied vacuously, since no oracle query is made. □

Corollary 4.14 (Quantified length leakage)**.** *Fix a padding profile (Section 3.6) with β reachable block sizes, and let the plaintext codepoint count n be drawn from an arbitrary distribution on the message space. Then*

$$I(n;\, |C|) \;\le\; \log_2 \beta,$$

and for a sequence of m ciphertexts, $I(n_1, \ldots, n_m;\, |C_1|, \ldots, |C_m|) \le m \log_2 \beta$, with no independence assumption on the n_i. Under the default bucket *profile $\beta = 13$, giving at most $\log_2 13 \approx 3.70$ bits per message; under* frame(F)*, $\beta = 1$ and the leakage is exactly zero. For AES-GCM and ChaCha20-Poly1305 on the same message space, $|C| = |M| + 16$ determines n exactly, so $I(n; |C|) = H(n)$ — up to 16 bits per message.*

Proof. By Lemma 3.13, $|C|$ is a function of $B(n)$, which takes at most β distinct values; hence $H(|C|) \le \log_2 \beta$ and $I(n; |C|) \le H(|C|) \le \log_2 \beta$. The m-message bound follows from subadditivity, $H(|C_1|, \ldots, |C_m|) \le \sum_i H(|C_i|)$. □

The next proposition is stated because it is the question a reader evaluating the cost of this construction will ask, and because its answer determines where the length-hiding property above actually comes from.

Proposition 4.15 (Expansion is length-neutral)**.** *Let the wire format induce a map f from padding buckets to ciphertext lengths, so that $|C| = f(B)$, and suppose f is public and injective. Then for every prior on the message space and every observer, the posterior distribution of the plaintext given $|C|$ is the same for every such f. In particular, replacing the token ceiling* MAX_NOISE_RUN $+ 1 = 20$ *and the* 8*-byte token width by* any *other constants — including* 1 *and* 1*, that is, deleting the arithmetic layer outright — leaves the leakage of Corollary 4.14 exactly unchanged.*

Proof. If f is public and injective then B and $f(B)$ are computable from one another by the observer, so the two generate the same σ-algebra and condition identically; the claim is then the data-processing equality for a bijection. The wire format of Section 3.9 has $f(B) = 48 + 8(B + 2)(\text{MAX_NOISE_RUN} + 1)$, strictly increasing in B and hence injective. □

Remark 4.16 (Coarsening, not inflation, is what hides length)**.** *Proposition 4.15 isolates the mechanism. Theorem 4.12 and Corollary 4.14 are consequences of the* many-to-one *padding map $n \mapsto B(n)$ of Section 3.5, which collapses 2^{16} possible codepoint counts onto* 13 *buckets. They are* not *consequences of the multi-prime token map, of the noise schedule, or of the fixed* 20*-tokens-per-real-token ceiling: each of those is injective, publicly invertible post-processing of B, and by Proposition 4.15 contributes exactly zero bits of length confidentiality. It follows that a reader whose sole objective is traffic-analysis resistance should tune the coarseness of $B(\cdot)$ (Section 3.6) and not the expansion factor —* frame(F) *reduces the leakage to zero, which no choice of expansion factor can do. Section 6 states what the arithmetic layer is retained for instead, and Section 6.1 gives the construction with that layer removed, for comparison.*

Measured leakage. Corollary 4.14 is an upper bound; the realised leakage depends on the length distribution, so we measure it. The harness `traffic_analysis_bench.py` in the reference distribution encrypts a grid of plaintext lengths under each profile and each comparison scheme and reports, for a uniform prior over 0–512 codepoints on a stride-8 grid ($H(n) = 6.02$ bits), the realised $I(n; |C|)$, the success rate of a maximum-a-posteriori adversary recovering n exactly from $|C|$, and the success rate of a two-class adversary distinguishing a short command (12–20 codepoints) from a full parameter set (300–500), where 50% is a coin flip.

Scheme	Distinct $\lvert C\rvert$	$I(n; \lvert C\rvert)$	Recover n	Two-class
NAPQES, bucket (default)	7	2.02 b	10.8%	100%
NAPQES, coarse(3)	3	0.94 b	4.6%	100%
NAPQES, frame(1024)	1	0.00 b	1.5%	50.0%
AES-GCM	65	6.02 b	100%	100%
ChaCha20-Poly1305	65	6.02 b	100%	100%

Table 2: Measured length leakage over a uniform prior on 0–512 codepoints. The two-class column is reported separately because it is not governed by the average-case bound: a coarse profile can leave $I(n; |C|)$ small while still separating two classes perfectly.

Two observations from Table 2 are worth stating explicitly, since neither is visible from Corollary 4.14 alone. First, the bound is not tight: the realised 2.02 bits under the default profile is well below $\log_2 13$, because a bounded length prior reaches only some of the buckets. Second, and more importantly, a *small* $I(n; |C|)$ does not imply resistance to a *specific* distinguisher. Under both bucket and coarse(3) the two-class adversary succeeds with certainty, since the two classes occupy disjoint bucket sets — $\{16, 32\}$ against $\{512\}$ under bucket, $\{16, 128\}$ against $\{1024\}$ under coarse(3) — whatever the expansion factor; only frame(F) with F above the longer class reduces it to a coin flip (the 50% for frame(1024) follows from 1024 exceeding both classes).

Deployments that need a particular pair of message populations to be indistinguishable must therefore choose F larger than both, and must not infer that result from the average-case bound.

4.4 INT-CTXT: Ciphertext Integrity

Definition 4.17 (INT-CTXT security)**.** *For a PPT adversary $\mathcal{A}$, define $\mathbf{Exp}^{\text{int-ctxt}}_{\mathcal{A}}(\lambda)$:*

1. $(\mathbf{k}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$*, and the challenger initialises an empty set $\mathcal{S}$.*
2. *$\mathcal{A}$ is given adaptive access to two oracles, interleaved in any order of its choosing:*
   - *$\mathcal{O}_{\mathsf{Enc}}(A, M)$ returns $c \leftarrow \mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M)$ and adds the pair (c, A) to $\mathcal{S}$. At most q such queries are made.*
   - *$\mathcal{O}_{\mathsf{Vf}}(A, c)$ computes $\mathsf{Dec}(\mathbf{k}, \mathsf{sk}, A, c)$ and returns the single bit $[\mathsf{Dec}(\cdot) \neq \perp]$ — the verification oracle reveals* only *acceptance or rejection, never the recovered plaintext. At most $q_v \geq 1$ such queries are made.*
3. *The experiment outputs* 1 *iff at some point $\mathcal{A}$ submits a* fresh *query $\mathcal{O}_{\mathsf{Vf}}(A^*, c^*)$ — one with $(c^*, A^*) \notin \mathcal{S}$* at the moment of submission *— that returns* 1*.*

The experiment does not halt on a successful forgery: $\mathcal{A}$ may continue querying both oracles, and wins if any *of its q_v verification queries is fresh and accepting. The advantage of $\mathcal{A}$ is* $\mathrm{Adv}^{\text{INT-CTXT}}_{\Pi}(\mathcal{A}) = \Pr[\mathbf{Exp}^{\text{int-ctxt}}_{\mathcal{A}}(\lambda) = 1]$*. Freshness is measured over the* pair (c, A)*, so a replay of a previously returned ciphertext under a different associated-data string counts as a forgery attempt, as it must for an AEAD scheme.*

Theorem 4.18 (INT-CTXT)**.** *Let $\mathcal{A}$ be any PPT adversary in the experiment of Definition 4.17, making at most q encryption queries and at most $q_v \geq 1$ adaptive verification queries. There exist PRF adversaries $\mathcal{B}_0, \mathcal{B}_2$ with similar running time such that*

$$\mathrm{Adv}^{\text{INT-CTXT}}_{\Pi}(\mathcal{A}) \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0) + \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q_v}{2^{256}}.$$

There is no q-dependent (encryption-query-count) term, since Case 2 below is resolved deterministically rather than probabilistically. The $\mathcal{B}_0$ term accounts for the one-query PRF hop that replaces $\mathsf{sk}_{\text{fmt}} = \text{HMAC}(\mathsf{sk}, \texttt{0x0B}\|\text{format_id})$ by a uniform 256-bit string (Definition 3.5); the $\mathcal{B}_2$ term is the subsequent PRF hop against $\text{HMAC}(\mathsf{sk}_{\text{fmt}}, \cdot)$ used in the proof below.

Proof. As in Theorem 4.8, we first apply the format-subkey PRF hop: replace $\mathsf{sk}_{\text{fmt}} = \text{HMAC}(\mathsf{sk}, \texttt{0x0B}\|\text{format_id}$ by an independent, uniform 256-bit string, at cost $\mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0)$ for some PRF adversary $\mathcal{B}_0$. Under the modified game, sk_{fmt} is now a uniform HMAC key independent of sk and $\mathbf{k}$, and every subsequent argument proceeds against this key.

Write a forgery attempt as $c^* = (N^*, \widetilde{B}^*, T^*)$ with tag input $x^* = \texttt{0x03}\|N^*\|\text{TagIn}(aad^*, \widetilde{B}^*)$ (Definition 3.6), the same quantity Dec step (4) recomputes.

Case 1: N^* was not used in any encryption query. Lemma 3.7 is a statement about the injectivity of the input encoding over its whole argument domain, so it applies to x^* even though $\mathcal{A}$, not the construction, chose the fields of x^*. By that lemma, x^* is distinct from every query-phase domain `0x03` input (different nonce at the fixed offset) and from every other domain's input, so x^* was never submitted to $\text{HMAC}(\mathsf{sk}_{\text{fmt}}, \cdot)$. Construct $\mathcal{B}_2$, given an oracle f (real or random), exactly as $\mathcal{B}_1$ in Lemma 4.9: sample $\mathbf{k}'$ independently, forward every HMAC call to f, and output "real" iff some forgery attempt's candidate tag matches $f(x^*)$. In the ideal world $f(x^*)$ is uniform and independent of $\mathcal{A}$'s view, so each of the q_v attempts succeeds by blind guessing with probability exactly 2^{-256}; a union bound gives $\Pr[\text{forge in Case 1}] \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + q_v/2^{256}$.

Case 2: $N^* = N_i$ for some query i. If $x^* \neq x_i$ for every such i (which, by injectivity of the length-prefixed AAD encoding, covers both $\widetilde{B}^* \neq \widetilde{B}_i$ and the $\widetilde{B}^* = \widetilde{B}_i \wedge aad^* \neq aad_i$ replay-with-new-AAD sub-case), x^* was never queried and Case 1's argument applies verbatim. If instead $x^* = x_i$ for some i, injectivity forces $aad^* = aad_i \wedge \widetilde{B}^* = \widetilde{B}_i$: if also $T^* = T_i$ then $(c^*, aad^*) = (c_i, aad_i)$, excluded by freshness; if $T^* \neq T_i$, the recomputed tag equals $T_i \neq T^*$ and Dec rejects unconditionally. Either sub-case contributes no successful, freshness-respecting forgery, and both branches are resolved deterministically via Lemma 3.7's injectivity — N^* is chosen adaptively by $\mathcal{A}$ after observing $N_1, \ldots, N_q$, so no probabilistic collision term applies to Case 2.

Combining both cases with the initial format-subkey PRF hop yields the stated bound. □

4.5 IND-CCA-det Security

Definition 4.19 (IND-CCA-det security)**.** *For a PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, define $\mathbf{Exp}^{\text{ind-cca-det}}_{\mathcal{A}}(\lambda)$ exactly as $\mathbf{Exp}^{\text{ind-cpa-det}}_{\mathcal{A}}(\lambda)$ of Definition 4.4, with these additions:*

- *In both phases $\mathcal{A}$ additionally has adaptive access to a decryption oracle $\mathcal{O}_{\mathsf{Dec}}(A, c) = \mathsf{Dec}(\mathbf{k}, \mathsf{sk}, A, c)$, returning the recovered plaintext or $\perp$, making at most q_d such queries in total.*
- *After the challenge c^* on associated data A^* has been issued, $\mathcal{A}_2$ may query $\mathcal{O}_{\mathsf{Dec}}$ on anything except the exact pair (A^*, c^*). In particular (A, c^*) for $A \neq A^*$ is permitted, as in the standard AEAD formulation.*

Restrictions (i) and (ii) of Definition 4.4 bind $\mathcal{A}$'s encryption-*oracle queries in both phases, and are not imposed on decryption queries. The advantage of $\mathcal{A}$ is* $\mathrm{Adv}^{\text{IND-CCA-det}}_{\Pi}(\mathcal{A}) = |\Pr[\mathbf{Exp}^{\text{ind-cca-det}}_{\mathcal{A}}(\lambda) = 1] - \frac{1}{2}|$.

Theorem 4.20 (IND-CCA-det)**.** *For any PPT adversary $\mathcal{A}$ in the experiment of Definition 4.19, making at most q encryption queries and q_d decryption queries, there exist PRF adversaries $\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}_2$ with similar running time such that*

$$\mathrm{Adv}^{\text{IND-CCA-det}}_{\Pi}(\mathcal{A}) \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0) + \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q^2}{2^{128}} + \frac{q_d}{2^{256}}.$$

The $\mathcal{B}_0$ term is the single-query PRF hop replacing $\mathsf{sk}_{\text{fmt}} = \text{HMAC}(\mathsf{sk}, \mathtt{0x0B} \| \text{format_id})$ (Definition 3.5) by a uniform 256-bit string, and is inherited from Theorems 4.8 and 4.18. Once sk_{fmt} is uniform, $\mathcal{B}_1$ and $\mathcal{B}_2$ are the same downstream PRF hops as in those two theorems (a single $\mathcal{B}_0$ term suffices, since Theorems 4.8 *and* 4.18*'s $\mathcal{B}_0$ hops target the identical* sk*-keyed HMAC call).*

Proof. The argument is the standard encrypt-then-authenticate composition of Bellare and Namprempre [16], but that result is stated for the unrestricted IND-CPA notion, which no deterministic scheme satisfies (Section 4.2). We therefore do not invoke it as a black box; we give the two game hops in full against Definitions 4.4, 4.17 and 4.19. Nothing beyond the restricted notions is used.

Before the game hops below, apply the format-subkey PRF hop of Theorem 4.8's proof once, replacing $\mathsf{sk}_{\text{fmt}} = \text{HMAC}(\mathsf{sk}, \mathtt{0x0B} \| \text{format_id})$ by an independent uniform 256-bit string at cost $\mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_0)$. Because both downstream reductions (to $\mathrm{Adv}^{\text{PRF}}(\mathcal{B}_1)$ via Theorem 4.8 and to $\mathrm{Adv}^{\text{PRF}}(\mathcal{B}_2)$ via Theorem 4.18) invoke exactly the same format-subkey hop against the same sk-keyed HMAC call, one $\mathcal{B}_0$ term suffices for the composition rather than two.

Game H_0 (real). The IND-CCA-det experiment of Definition 4.19, verbatim.

Game H_1 (table-backed decryption). As H_0, except that the challenger maintains a table T recording $(c, A) \mapsto M$ for every encryption-oracle answer c on input (A, M), together with $(c^*, A^*) \mapsto m_b$ once the challenge is issued, and answers a decryption query (A, c) with $T[(c, A)]$ if that entry exists and with $\perp$ otherwise.

Transition. Let D be the event that $\mathcal{A}$ submits a decryption query (A, c) with $(c, A) \notin T$ for which $\mathsf{Dec}(\mathbf{k}, \mathsf{sk}, A, c) \neq \perp$. On every query with $(c, A) \in T$ the two games agree: H_1 returns the recorded plaintext, and H_0 returns the same value by correctness (Proposition 3.18). On every query with $(c, A) \notin T$ the two games agree unless D occurs. Hence H_0 and H_1 are identical until D, and $|\Pr[H_0{=}1] - \Pr[H_1{=}1]| \leq \Pr[D]$.

Bounding $\Pr[D]$. Construct an INT-CTXT adversary $\mathcal{A}'$ (Definition 4.17) that runs $\mathcal{A}$, maintaining its own copy of T. It forwards each of $\mathcal{A}$'s q encryption queries to $\mathcal{O}_{\mathsf{Enc}}$, recording the answers in T; it samples b itself and obtains the challenge c^* by a $(q{+}1)$-th query $\mathcal{O}_{\mathsf{Enc}}(A^*, m_b)$, recording $(c^*, A^*) \mapsto m_b$. A decryption query (A, c) with $(c, A) \in T$ is answered from T; one with $(c, A) \notin T$ is forwarded to $\mathcal{O}_{\mathsf{Vf}}(A, c)$, and $\mathcal{A}'$ answers $\perp$ to $\mathcal{A}$ if the oracle returns 0, halting immediately if it returns 1. Note that $\mathcal{A}'$ only ever needs the verification oracle's single bit, never a decrypted plaintext, so the simulation is compatible with the bit-only oracle of Definition 4.17. Every query $\mathcal{A}'$ forwards to $\mathcal{O}_{\mathsf{Vf}}$ is fresh, because T contains exactly the pairs the encryption oracle returned. The simulation is perfect up to and including the first occurrence of D, which is all that is required, and D occurring is precisely $\mathcal{A}'$ winning. With $q_v = q_d$ verification queries, Theorem 4.18 gives $\Pr[D] \leq \mathrm{Adv}^{\text{INT-CTXT}}_{\Pi}(\mathcal{A}') \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + q_d/2^{256}$. The extra $(q{+}1)$-th encryption query costs nothing, since that bound carries no encryption-query-count term.

Bounding $|\Pr[H_1{=}1] - \frac{1}{2}|$. Construct an IND-CPA-det adversary $\mathcal{A}''$ (Definition 4.4) that runs $\mathcal{A}$, forwards its encryption queries to its own encryption oracle while recording T, forwards (A^*, m_0, m_1) as its own challenge and relays the challenger's c^*, and answers decryption queries from T, returning $\perp$ for entries it does not hold. This is exactly H_1's oracle: the only table entry $\mathcal{A}''$ cannot populate is $(c^*, A^*) \mapsto m_b$, and that entry is never read, since Definition 4.19 forbids $\mathcal{A}$ the pair (A^*, c^*). Any permitted query (A, c^*) with $A \neq A^*$ is a distinct table key, absent from T in H_1 as well, so both answer $\perp$. $\mathcal{A}''$ inherits restrictions (i) and (ii) from $\mathcal{A}$ and makes the same q encryption queries, so by Theorem 4.8, $|\Pr[H_1{=}1] - \frac{1}{2}| \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + q^2/2^{128}$.

Combining the two bounds yields the stated inequality. □

5 Post-Quantum Analysis

SHA-256 and Grover's algorithm. Grover's algorithm [8] provides a quadratic speedup for unstructured search, requiring $\approx 2^{n/2}$ oracle queries against an n-bit target. Applied to SHA-256 preimage search, this reduces the nominal security level from 256 to approximately 128 bits. This 2^{128} figure is a bound on *sequential query depth*, not a parallelizable work-factor: the Bennett–Bernstein–Brassard–Vazirani (BBBV) optimality result [9] shows no quantum algorithm can search an unstructured space of size N in fewer than $\Omega(\sqrt{N})$ oracle queries, and Zalka [10] shows that distributing Grover's search across S machines yields only a $\sqrt{S}$ speedup — the depth stays $\approx 2^{n/2}/\sqrt{S}$ and the total work is unchanged, unlike classical brute force, which parallelizes linearly. NIST's transition guidance [7] deprecates 112-bit security after 2030 and sets 128 bits as the minimum from that point; at 2^{128} sequential oracle queries this construction meets that post-2030 minimum, rather than merely exceeding the 112-bit level being retired.

No Shor-applicable structure. NAPQES does not use discrete logarithms, integer factorisation, or any algebraic group structure, so Shor's algorithm provides no speedup against it. This is a property NAPQES *shares* with AES-GCM, ChaCha20-Poly1305 and Ascon, all of which are likewise symmetric and likewise unaffected by Shor; it is recorded here to forestall the common misreading that a symmetric scheme's arithmetic content makes it quantum-vulnerable, and it is not offered as a differentiator. The differentiator claimed in this paper is the absence of a polynomial authenticator and the resulting absence of the nonce-reuse key-recovery mode of Section 2.

Two separate key-space figures. Because sk is the sole HMAC key and $\mathbf{k}$ is used only by the un-keyed arithmetic layer (Remark 3.4), the two secrets have different, non-interchangeable security roles and are best reported separately rather than as one conflated figure:

- sk: uniform 256 bits, classical exhaustive search 2^{256}, post-Grover sequential depth $\approx 2^{128}$ — this is the figure relevant to Assumption 4.2 and to every theorem of Section 4.
- $\mathbf{k}$: $\approx 2^{256.97}$ ordered-tuple key-space size at the default $K = 13$ (post-Grover $\approx 2^{128.49}$). This figure is reported for completeness only. It is *not* relevant to any theorem in Section 4, none of which depend on $H_\infty(\mathbf{k})$ (Remark 3.4), nor to the length-hiding property of Section 4.3, which is independent of the arithmetic layer entirely (Remark 4.16). What that layer is retained for is set out in Section 6.

Both figures are non-parallelizable sequential-depth bounds by the same BBBV [9]/Zalka [10] argument. Section 12 discusses the practical consequence of this asymmetry: protecting sk is necessary and sufficient for confidentiality and integrity; protecting $\mathbf{k}$ in addition provides no independent confidentiality margin once sk is compromised.

6 What the Arithmetic Layer Does and Does Not Buy

The arithmetic layer — the multi-prime token map $c \mapsto c \cdot k_j + a$, the noise schedule, and the 20-token ceiling — accounts for essentially the whole cost of this construction, and for none of its theorems. A reader deciding whether to deploy NAPQES is entitled to a direct statement of what that cost purchases. We give one, separated into what is proved, what is argued but not proved, and what is not claimed at all.

Proved: the noise layer is length-neutral. Lemma 3.13 is a property *of* the arithmetic layer, but a defensive one: the MAX_NOISE_RUN cap together with the ceiling-padding step exists to ensure that the noise schedule — which is derived from the message-dependent synthetic nonce (Definition 3.12) — does not itself introduce a message-dependent length. Without the cap and ceiling, the natural token count would vary with the message even at a fixed bucket, and an observer collecting several encryptions of one message under varying AAD could average that variance away and recover the bucket, and more. The cap and ceiling remove that channel entirely. This is a genuine contribution of the layer's design, but it is the removal of a leak that the layer would otherwise create; it is not an improvement on a construction that has no noise layer to begin with.

Argued, not proved: degradation under nonce collision. Lemma 4.7 bounds the probability of two distinct (A, M) pairs sharing a nonce by $\mathrm{Adv}^{\mathrm{PRF}} + 2^{-128}$, so what follows lies outside the model of Section 4 and is *not* claimed as a security property. Should such a collision nevertheless occur, the two ciphertexts share a keystream and the adversary obtains $B_1 \oplus B_2$. In a construction that masks padded codepoints directly, this is a textbook two-time pad on the plaintexts, recoverable by standard crib-dragging. Here it is the XOR of two token

blobs: the noise and filler positions, being derived from the shared nonce alone, cancel to zero and are thereby revealed, while each real position yields the XOR of two affine images $c \cdot k_j + a$ and $c' \cdot k_j + a$ sharing an unknown multiplier k_j and a common addend. Recovering codepoints therefore requires additionally resolving $\mathbf{k}$. We record this as strictly more work for the adversary than a plain two-time pad, and as nothing more: it is substantial plaintext leakage in both cases, it admits no bound we are able to state, and it must not be read as a defence.

Not claimed. Three claims that this layer might be expected to support are not made anywhere in this paper, and we state their absence explicitly.

- *No content confidentiality.* The one-time-pad step of Lemma 4.10 is established from domain `0x07` alone and never inspects the token blob it masks. The arithmetic layer contributes nothing to Theorem 4.8, Theorem 4.18, or Theorem 4.20, and removing it changes none of their statements or bounds (Proposition 6.1).
- *No length hiding.* By Proposition 4.15 the expansion factor is a public injective function of the padding bucket and contributes exactly zero bits of length confidentiality; all of the property proved in Section 4.3 comes from the coarseness of $B(\cdot)$, which survives the removal of the arithmetic layer untouched (Remark 4.16).
- *No dependence on K or on $H_\infty(\mathbf{k})$.* K is an interoperability parameter, not a security parameter (Remark 3.4), and $\mathbf{k}$ adds no confidentiality margin once `sk` is known (Section 12).

6.1 The Construction Without the Arithmetic Layer

To make the trade-off precise rather than rhetorical, we specify the same construction with the arithmetic layer deleted and compare the two. Define *NAPQES-L* identically to Definition 3.14 — same key generation, same domain separation, same padding and padding profiles, same synthetic nonce, same domain-`0x07` keystream, same domain-`0x03` tag over the same TagIn — except that the token-emission loop of Section 3.7 is replaced by the identity map on padded codepoints, each serialised as 3 bytes big-endian. Domains `0x00`, `0x01`, `0x02`, `0x04` and `0x05` are then unused, $\mathbf{k}$ and K disappear from the scheme, and

$$|C|_{\text{NAPQES-L}} \;=\; 16 + 3(B+2) + 32 \;=\; 48 + 3(B+2) \text{ bytes},$$

against $|C| = 48 + 160(B+2)$ for NAPQES as specified; at the minimum bucket $B = 16$ this is 102 bytes against 2928.

Proposition 6.1 (The theorems are indifferent to the arithmetic layer)**.** *Theorem 4.8, Theorem 4.12, Theorem 4.18 and Theorem 4.20 hold for NAPQES-L with identical statements and identical bounds, by the same proofs.*

Proof. Each proof uses exactly four facts about the masked payload: that domain separation holds (Lemma 3.7), which is unchanged; that the nonce-collision bound holds (Lemma 4.7), which depends only on domain `0x0A` and is unchanged; that the payload is assembled entirely from domains other than `0x07`, so that the one-time-pad step of Lemma 4.10 applies — in NAPQES-L the payload is the padded codepoint sequence, built from the plaintext and domain `0x06` alone, so this still holds; and that the payload length is a function of B alone, which for NAPQES-L is immediate, since it is exactly $3(B+2)$ bytes. No proof step examines the token map, the noise schedule, or $\mathbf{k}$. □

NAPQES-L is stated for comparison only. It is **not** part of this specification, has no wire format, no test vectors and no reference implementation, and is not interoperable with NAPQES. We include it because the honest summary of Table 3 is that the arithmetic layer costs a factor of ≈53 in bandwidth and buys no theorem; a deployment selecting NAPQES over NAPQES-L is selecting the beyond-model margin described above, and should do so deliberately.

	NAPQES (specified)	NAPQES-L
IND-CPA-det, INT-CTXT, IND-CCA-det	identical bounds	identical bounds
LH-IND-CPA-det (Thm. 4.12)	holds	holds
Length leakage (Cor. 4.14)	$\leq \log_2 \beta$	$\leq \log_2 \beta$
Ciphertext size, bucket B	$48 + 160(B + 2)$ B	$48 + 3(B + 2)$ B
Minimum ciphertext	2928 B	102 B
Secrets	$\mathbf{k}$ and sk	sk only
Trial-division caveat (§12)	applies	vacuous
Nonce-collision residue	affine, $\mathbf{k}$ unknown	two-time pad

Table 3: NAPQES as specified against the same construction with the arithmetic layer removed. No theorem of Section 4 changes.

7 Comparison with AES-GCM, ChaCha20-Poly1305, and Ascon

Property	NAPQES	AES-GCM	ChaCha20-Poly1305	Ascon
Primitive basis	HMAC-SHA256	AES + GHASH	ChaCha20 + Poly1305	Ascon permutation
FIPS-approved primitives only	**Yes**	Yes	No	Yes [6]
Approved as a mode/algorithm	No	Yes	No	Yes [6]
Shor-applicable structure	No	No	No	No
Forbidden Attack risk	**No**	Yes (nonce reuse)	Yes (nonce reuse)	No
Nonce-reuse key recovery	**No** (Definition 3.12, disclosed message-equality only)	Yes (GCM key)	Yes (Poly1305 key)	Partial
Ciphertext expansion	160 B per padded codepoint	1×	1×	1×
Length leakage per message	≤ 3.70 bits (0 with frame)	$= H(n)$	$= H(n)$	$= H(n)$
Key size	$5K + 32$ bytes ($\mathbf{k}$+sk; 97 at $K = 13$)	32 bytes	32 bytes	16 bytes

Table 4: Comparison of NAPQES with established AEAD schemes. The two FIPS rows are distinct and must not be conflated: NAPQES uses only an approved primitive (HMAC-SHA256, FIPS 198-1/180-4), but NAPQES *itself* is not an approved algorithm or mode. In a FIPS 140-3 validation it would fall outside the approved-mode boundary and would be reported as a non-approved algorithm, notwithstanding the provenance of its primitive.

The primary trade-off is ciphertext expansion. The token-emission loop caps consecutive noise tokens at MAX_NOISE_RUN = 19 (Section 3.7) and pads every message to exactly $R \cdot (\text{MAX_NOISE_RUN} + 1) = 20R$ tokens of 8 bytes each, so the expansion is a hard, deterministic 160 bytes per padded codepoint — never a range, and never dependent on the per-message noise realisation (Lemma 3.13). We do not offer this expansion as a traffic-analysis countermeasure; by Proposition 4.15 it provides none, and Section 6.1 gives the same construction without it, under which every theorem of Section 4 continues to hold with identical bounds. What NAPQES does provide against a length-observing adversary, and AES-GCM and ChaCha20-Poly1305 do not (Proposition 4.13), is the length-hiding property of Section 4.3, which follows from the padding profile alone and is tunable down to zero leakage via $\mathsf{frame}(F)$ (Section 3.6).

Unlike AES-GCM and ChaCha20-Poly1305, where nonce reuse degrades confidentiality but does not expose the key, NAPQES's synthetic, message-bound nonce (Definition 3.12) removes nonce selection from the caller entirely, so there is no DRBG-failure or repeated-counter path to reuse; re-querying with an *identical* (A, M) pair reproduces the identical ciphertext deterministically, disclosing only plaintext equality — the standard, disclosed MRAE trade-off (cf. AES-GCM-SIV [4]), not a confidentiality or key-recovery break.

This does *not* eliminate the birthday bound. Two distinct (A, M) pairs collide on the truncated 128-bit nonce with probability 2^{-128} each (Lemma 4.7), and the $q^2/2^{128}$ term in Theorems 4.8 and 4.20 is exactly the union bound over the $\binom{q}{2}$ pairs — an ordinary birthday term. NAPQES therefore carries a data-complexity limit of roughly 2^{64} messages per key, of the same order as any 128-bit-nonce AEAD, and the practical difference from AES-GCM is in *how* a nonce collision arises (a PRF collision on distinct inputs, never a caller or DRBG error) and in what it costs when it does, not in whether the birthday bound applies. Section 12 states the recommended operational cap.

8 Implementation

8.1 Python Reference Implementation

The Python reference implementation (`napqes.py`) is pure Python 3.10+, with no dependencies beyond the standard library (`hmac`, `hashlib`, `secrets`, `base64`). Key generation and the AEAD entry points are `generate_v8_key`, `encrypt_bytes_v8`, and `decrypt_bytes_v8`. All KAT vectors are derived from this implementation.

8.2 Rust Implementation

A Rust port (`rust/src/lib.rs`) mirrors the Python implementation (`generate_v8_key`, `encrypt_bytes_v8`, `decrypt_bytes_v8`). Unit tests cover round-trip correctness, tamper/AAD-mismatch rejection, same-message determinism (the disclosed MRAE trade-off), distinct-message nonce distinctness, and ciphertext-length determinism across varied AAD and keys (Lemma 3.13). A validated key wrapper (`NapqesKey`) runs key validation once at construction and wipes the key on drop.

The crate additionally exports the legacy v7 format: decryptors kept for archived ciphertexts, and `#[deprecated]` encryptors, compiled only with the `legacy-v7` feature, that draw a CSPRNG nonce, use an uncapped noise loop (so ciphertext length varies with the nonce) and an IEEE-754 `f64` noise threshold. None of the results of Section 4 apply to v7. The crate's OT-frame wrapper (`rust/src/ot_frame.rs`) seals every frame with the v8 block format under a validated key whose prime tuple and `sk` are derived from the key-establishment shared secret with distinct HKDF labels (CVF-V5-7); no in-tree component encrypts with v7.

8.3 C Implementation

A C port (`C/napqes.c`) mirrors the same construction (`napqes_encrypt_bytes_v8`, `napqes_decrypt_bytes_v8`

8.4 Constant-Time Considerations

Threat model. Every claim in Section 4 is made in the standard black-box model, in which the adversary sees only oracle inputs and outputs. Side channels — wall-clock time, cache state, branch predictor state, power, electromagnetic emanation — are explicitly outside that model, and none of the theorems in this paper says anything about them. This subsection records what the reference implementations do and do not do; it is not a claim of side-channel resistance, and the phrase "compared in constant time" in Dec step (5) (Definition 3.14) should be read as describing exactly one operation, not the algorithm as a whole.

What is constant-time. Authentication tag comparison uses `hmac.compare_digest` (Python), `ConstantTimeEq` from the `subtle` crate (Rust), and an OR-accumulator byte-difference loop (C), so that comparison timing does not depend on the position of the first differing tag byte. This is the property that matters for forgery attempts, since it prevents byte-at-a-time tag search. The C accumulator is not declared `volatile`, so nothing in the C standard prevents a sufficiently

aggressive compiler from reintroducing an early exit; this is the usual caveat for hand-rolled constant-time C and is not addressed here.

What is not. The token-emission and token-inversion loops are *not* constant-time, and are not intended to be:

- The number of loop iterations depends on the realised noise pattern, which is a deterministic function of (sk, N). The total ciphertext length is padded to a fixed ceiling (Lemma 3.13) and so leaks nothing, but the *time* to produce or consume it varies with the number of noise tokens actually emitted before the run cap engages.
- Both directions use arbitrary-precision or 64-bit integer multiplication and division by a key element k, and add or subtract the secret-derived addend a; both k (selected by $real_idx \bmod K$) and a are secret. Division latency is data-dependent on many microarchitectures.
- Key validation (primality by trial division, range and distinctness checks) runs in time that depends on the secret key elements. The bare-slice entry points of all three ports repeat it on every call; the Rust `NapqesKey` wrapper performs it once, at construction. The Rust v7 decryptors perform it once per call and only *after* the tag has verified, so a forged or truncated ciphertext is rejected after one HMAC evaluation and never reaches the key-dependent validation (CVF-V5-12).
- The retained v7 encryptors (Section 8) compute their noise threshold in IEEE-754 `f64`; they are outside Π and carry no timing or reproducibility claim.
- The padding loop of Section 3.5 performs $B - n$ derivations, so encryption time depends on the *exact* codepoint count n, not merely on its padding bucket B. The padding-bucket caveat below concedes a $\log_2 13 \approx 3.70$-bit length leak through ciphertext *length*; a timing channel can in principle recover more than that.
- The Python reference implementation offers no timing guarantees of any kind: it uses `int`, list resizing, and dictionary lookups throughout.
- The structural checks at Dec steps (6), (7) and (8) (Remark 3.17) — the bucket check, the per-token guards $t \geq a$, $(t - a) \equiv 0 \pmod{k}$ and the scalar-value check, and the length-prefix checks — reject at different points and in different time. They are reachable only after the tag has verified, i.e. only by a party already holding sk, so this difference is not an oracle for an outside adversary.

An attacker with a precise timing channel against a decrypting party may therefore learn something about the noise realisation, and hence about $\theta(N)$ and indirectly about N; N is transmitted in the clear in any case, so this particular leak is not by itself a break, but no bound is offered on what else such a channel exposes. A dudect timing study of the Rust port (`rust/examples/dudect_harness.rs`) is reported in `docs/DUDECT_ATTESTATION.md` (round 5, CVF-V5-27). Its primary evidence times the tag comparator *alone* on operands differing in the first versus the last byte: $|t| = 0.79$ after 7.6×10^6 samples, while a positive control (a deliberately early-exit comparator measured the same way) reaches $|t| > 4{,}600$ within a minute, showing the test has the power to detect such a leak; CI repeats the comparator test on every change with a fixed threshold of 4.5. A second measurement of the whole `decrypt_bytes_v8_key` call ($|t| = 1.96$ after 1.3×10^7 samples) is low-power evidence only, since the comparator is about 0.1% of that call. Both cover the tag comparison only. Deployments in which the adversary can measure decryption time on attacker-influenced inputs should treat this as an open gap and are directed to Section 12.

9 NIST SP 800-22 Statistical Analysis

We applied the full NIST SP 800-22 Rev. 1a Statistical Test Suite [18] to 5×10^7 bits of NAPQES ciphertext produced by Enc as specified here, encrypting 480-codepoint slices of a cycling printable-ASCII corpus under a 10-element prime key, a fixed sk, and a distinct per-slice AAD (without it the deterministic Enc repeats its output whenever the corpus slice repeats, and the suite correctly rejects the repetition). All 14 implemented tests (Overlapping Template Matching is not implemented; CVF-V5-24), 40 scored sub-results, passed at the $\alpha = 0.01$ significance level; the smallest p-value was 0.016. Results are reproducible with `rust/src/bin/sts.rs` and recorded in `sts_report.json` and `docs/STS_ATTESTATION.md`.

10 Known-Answer Tests

The corpus for the scheme specified in this paper is `tests/kat/v8_vectors.json`. It contains 37 vectors: 23 positive (ids `W001`–`W023`) covering the empty message, a single character, the block boundaries at 15, 16 and 32 codepoints, 1-element, 10-element and (`W023`) default 13-element prime tuples $\mathbf{k}$, the last with elements 9,900,047, 12,000,017 and 14,999,981 above the $\mathcal{P}_1$ bound, empty, binary and 64-byte associated data (the last pinning the 8-byte be8 AAD length prefix of Definition 3.6), mixed-case punctuation, the coarse(3), coarse(12) and frame(1024) padding profiles (`W013`–`W015`, the last at the largest admissible $n = F-1$), coarse(3) on both sides of its first stride boundary ($n = 15 \mapsto B = 16$ and $n = 16 \mapsto B = 128$, `W020`–`W021`), and non-ASCII messages spanning U+0080–U+00FF, the BMP either side of the surrogate block, the supplementary planes up to U+10FFFF (`W016`–`W019`) and the top of the BMP, U+FFFE and U+FFFF (`W022`); 12 negative; and two encryption-side negatives: `W-E01`, frame(1024) at $n = 1024$, which every port must refuse to encrypt, pinning the strict inequality $n < F$; and `W-E02`, a key containing 15,000,017, the smallest prime above the bound of $\mathcal{P}$, which every port must reject. The corpus also records the inclusive interval of $\mathcal{P}$ it pins as a top-level `prime_interval` field. Of the negative vectors, `W-N01`–`W-N05` exercise authentication (flipped tag byte, flipped masked-blob byte, wrong A, wrong sk) and the 48-byte parse floor; `W-N06` has an illegal token count and is rejected at Dec step (3) before the tag is computed; and `W-N07`–`W-N11` are *validly tagged* but structurally malformed and therefore reach the post-authentication checks of Dec: a real-token count that is a multiple of MAX_NOISE_RUN + 1 but is not $B+2$ for any reachable bucket B; a padded length prefix claiming more codepoints than the padded buffer holds; a real token with $t < a$ (`W-N09`) and one with $(t - a) \bmod k \neq 0$ (`W-N10`); a non-canonical length prefix $(0, 300)$ in place of $(1, 44)$ (`W-N11`); and a first real codepoint that decodes to the surrogate U+D800 (`W-N12`). Each positive vector fixes $(\mathbf{k}, \mathsf{sk}, A, M)$ and the expected ciphertext; because Enc is a pure function of its inputs (Definition 3.14), no nonce field appears and none is needed. Conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector. Each vector carries $\mathbf{k}$ as a JSON array of its prime elements in decimal, in tuple order (`key`), and sk, A and every ciphertext as lowercase hex (`sk_hex`, `aad_hex`, `ciphertext_hex`, `tampered_hex`). The SHA-256 of the file as committed (LF line endings) is `2eafeb4d98f87419a11031a194c8b2c2 5acbf8c57997f058cd26724635edfc0b`. The generator `tests/gen_kats_v8.py` regenerates the file and checks that every negative vector fails in the reference for its stated reason; its `--check` mode establishes only that the Python output is reproducible. Cross-port agreement is established by the consumers `tests/test_kats.py`, `rust/src/kat_cross_check.rs` and `C/test_kats.c`, which reproduce every positive ciphertext byte for byte, check every rejection reason, and pin the corpus size (the C byte API covers codepoints up to U+00FF). The streaming corpus `tests/kat/v8_stream_vectors.json` (11 positive, 3 negative with re-tagged non-canonical sentinels) is produced by `tests/gen_kats_v8_stream.py`.

A second corpus, `tests/kat/v6_vectors.json` (37 vectors), is retained in the repository for the earlier v7 wire format, which keys its derivations by key_bytes($\mathbf{k}$) rather than by sk and

whose vectors fix a per-vector nonce (the public v7 encryptors draw it from a CSPRNG; the fixed-nonce entry is a test-only helper); it is *not* the scheme analysed here; its vectors are outside the scope of this paper and none of the results of Section 4 are asserted of them.

Both corpora, together with the regression suites across the Python, Rust and C implementations, also check Proposition 3.18 empirically. As stated in Section 1, these are evidence of cross-implementation agreement only; no security claim in this paper rests on them.

11 Fuzz Testing

Two fuzz harnesses are provided. The Python harness (`tests/fuzz_atheris.py`) targets the legacy decoders, runs on Windows only in a seeded random mode without coverage guidance, and has no recorded coverage-guided run; it is not cited as evidence for Π (corrected in round 5, CVF-V5-26). The Rust harness (`rust/fuzz/`, using `cargo-fuzz`) has four targets: `decode_bytes` derives a valid ciphertext under any of the three padding profiles from the fuzzer input, mutates its body or nonce and *re-tags* it, so mutated inputs pass authentication and reach the post-authentication checks of Dec; `stream_decode` does the same for the experimental streaming format, re-tagging every chunk and the sentinel; `unauth_decode` feeds the unauthenticated token-slice decoder structurally valid $c \cdot k + a$ token streams with one mutation each; and `validate_key` feeds arbitrary key tuples to every entry point. `docs/FUZZ_ATTESTATION.md` records a ten-minute run of each target under AddressSanitizer at commit `471501e`, with no crash or panic (between 5.9×10^3 and 3.8×10^5 executions per target). This is a smoke test, not a saturation campaign: v8 ciphertexts are large, so the decoder targets execute at only 9 to 46 inputs per second.

12 Known Caveats

Plaintext length cap. The block padding scheme stores the plaintext length as a 2-codepoint big-endian prefix (two real tokens, occupying up to 320 bytes on the wire), capping messages at 65,535 codepoints. Exceeding the cap raises an error immediately in every v8 entry point; there is no silent truncation. Callers requiring larger messages must split at the application layer.

Ciphertext expansion. Fixed at exactly 20 tokens, i.e. 160 bytes, per padded codepoint (Section 7). The expansion is deterministic rather than message-dependent, which is what Lemma 3.13 requires, but it is *not* a traffic-analysis countermeasure: by Proposition 4.15 it contributes zero bits of length confidentiality. It is a cost of the arithmetic layer, whose retention is argued and bounded in Section 6 and quantified against the same construction without that layer in Section 6.1.

Padding length-bucket leakage. Ciphertext length reveals the padding bucket, and nothing else about the plaintext (Lemma 3.13). Under the default bucket profile there are 13 reachable buckets $\{16, 32, \ldots, 65536\}$, so the leakage is at most $\log_2 13 \approx 3.70$ bits per message (Corollary 4.14) — it does not reveal the exact codepoint count within a bucket, and, because the token ceiling is fixed per bucket, it does not vary with sk, the nonce, or the associated data, so no averaging attack across repeated encryptions of one message under varying AAD can extract more than the bucket itself. Deployments requiring strictly less than this should select a coarser padding profile, and those requiring zero length leakage should select frame(F) (Section 3.6), which needs no change to the wire format and no cooperation from the decryptor; no external fixed-frame transport is required.

Scope of sk and k compromise. sk is the sole cryptographic secret in this construction: one HMAC call keyed by sk (the domain-`0x0B` format-subkey derivation of Definition 3.5)

fixes sk_{fmt}, and every other HMAC domain derivation — the noise schedule, the padding, the per-position addends, the keystream, and the tag — is keyed by sk_{fmt} (Section 3.3). Consequently, an adversary who compromises sk but not $\mathbf{k}$ derives sk_{fmt} in one HMAC evaluation and can then recover any real token's plaintext codepoint directly: the per-position addend's raw HMAC output depends only on sk_{fmt}, so for a given token the adversary can, for each of the 892,206 candidate primes in $\mathcal{P}$, compute the corresponding addend and check whether the resulting quotient is an exact integer in a valid codepoint range — an $\mathcal{O}(|\mathcal{P}|)$ trial-division search that succeeds in microseconds. $\mathbf{k}$'s secrecy therefore adds **no** independent confidentiality margin once sk is known: compromise of sk alone is equivalent to a full break, regardless of whether $\mathbf{k}$ also leaked. Operationally, $\mathbf{k}$ and sk must therefore be protected with equal rigor, as a single unit — not because splitting them across separate storage or trust domains buys compartmentalized defense-in-depth (it does not), but because sk compromise alone is already fatal. This is consistent with, and does not weaken, the security theorems of Section 4, which assume the entire key material is secret from the adversary, exactly as for any symmetric AEAD scheme (a leaked AES-256 key is likewise a full break of AES-GCM).

Key lifetime. $\mathbf{k}$, sk and the derived sk_{fmt} should be erased as soon as they are no longer needed, with writes the compiler cannot elide. The Rust port wipes sk_{fmt} on every exit path of each entry point, and its `NapqesKey` wrapper wipes $\mathbf{k}$ and sk on drop. This paper makes no side-channel or memory-disclosure claim.

Message domain and the C port. The message space $\mathcal{M}$ excludes the surrogate range $[\mathtt{U+D800}, \mathtt{U+DFFF}]$, since the synthetic-nonce derivation hashes the UTF-8 encoding of M and UTF-8 is undefined there. Rust enforces this by type and Python rejects surrogates explicitly. The C port (`C/napqes.c`) instead maps each *byte* of a NUL-terminated `char *` to one codepoint (Remark 3.16), so it agrees with the other two ports on codepoints up to U+00FF only and cannot represent wider codepoints or an embedded NUL. Cross-implementation interoperability is therefore claimed for all of $\mathcal{M}$ between Python and Rust, and for the U+0000–U+00FF subset with C; the KAT corpus of Section 10 covers both (`W016`–`W019`). Fixing the C port to be codepoint-oriented would change no wire format but is not done here.

Per-key data limit. The $q^2/2^{128}$ term in Theorems 4.8 and 4.20 is an ordinary birthday bound over the 128-bit synthetic nonce, so NAPQES has a data-complexity limit of roughly 2^{64} encryptions per key, the same order as any 128-bit-nonce AEAD. Synthetic nonce derivation removes the *caller-error and DRBG-failure* routes to nonce reuse (Definition 3.12); it does not remove the birthday bound, and Section 7 should not be read as claiming otherwise. We recommend an operational cap of $q \leq 2^{48}$ encryptions per $(\mathbf{k}, \mathsf{sk})$ pair, at which the term is 2^{-32}, followed by rekeying. Neither the reference implementations nor the wire format enforce this cap; it is a deployment obligation.

Timing side channels. The security theorems of Section 4 are black-box statements and cover no side channel. Only the tag comparison is written to run in constant time; the token-emission and token-inversion loops iterate a noise-dependent number of times and perform integer division by a key element, both of which are data-dependent on real hardware, and the Python reference implementation offers no timing guarantees at all. Section 8.4 states precisely what is and is not covered. Deployments in which an adversary can measure encryption or decryption time on inputs it influences should treat this as an unresolved gap; a TVLA or dudect study of the full encode/decode path, rather than of the tag comparison alone, remains future work.

13 Conclusion

We have presented NAPQES, an AEAD scheme whose entire security reduces to the pseudorandomness of HMAC-SHA256. The construction relies solely on FIPS-approved primitives, and keys two independently sampled secrets — an arithmetic-layer prime tuple $\mathbf{k}$ and a uniform 256-bit HMAC key sk — together with a synthetic, message-bound nonce that removes nonce selection from the caller and degrades identical-message reuse only to disclosed plaintext equality. Like every symmetric AEAD scheme it is unaffected by Shor's algorithm; what distinguishes it from AES-GCM and ChaCha20-Poly1305 is the absence of a polynomial authenticator and hence of the nonce-reuse key-recovery mode that follows from one. We prove IND-CPA-det, INT-CTXT, and IND-CCA-det security under the standard, uniform-key PRF assumption on HMAC-SHA256, with no key-entropy term and no simulation gap, and prove that ciphertext length is a deterministic function of the padded-length bucket alone. Against a length-observing adversary we prove the strictly stronger LH-IND-CPA-det notion (Theorem 4.12) at the same bound, bound the resulting leakage at $\log_2 13 \approx 3.70$ bits per message and zero under the frame padding profile (Corollary 4.14), and exhibit the zero-query adversary that breaks the same notion outright for AES-GCM and ChaCha20-Poly1305 (Proposition 4.13). We also show that this property follows from the padding profile alone: the arithmetic layer contributes nothing to it (Proposition 4.15), and removing that layer changes no theorem in this paper (Proposition 6.1). Separately from the proofs, the Python, Rust and C implementations agree on the cross-implementation KAT corpora of Section 10, the ciphertext output passes the NIST SP 800-22 Rev. 1a suite (Section 9), and fuzz harnesses for the decoders are provided (Section 11); these are evidence of implementation agreement, statistical quality and decoder robustness respectively, and none of them is evidence for a security claim, all of which rest on the theorems alone.

Remaining open limitations are documented in Section 12: the fixed 160-byte-per-codepoint ciphertext expansion of the noise-injection design, which buys no theorem and is retained for the beyond-model reasons set out in Section 6, the 16-bit plaintext length cap, the residual padding length-bucket leak, and the scope of what sk and $\mathbf{k}$ compromise each imply. Future work includes a TVLA side-channel analysis of the Rust implementation's full encode/decode path (only its tag comparison is constant-time today), a FIPS 140-3 module boundary definition, and a performance evaluation on embedded ARM Cortex-M4 and RISC-V targets.

References

[1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.

[2] Y. Nir, A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, June 2018. (Obsoletes RFC 7539.)

[3] D. Harkins, *Synthetic Initialization Vector (SIV) Authenticated Encryption Using the Advanced Encryption Standard (AES)*, RFC 5297, October 2008.

[4] S. Gueron, A. Langley, Y. Lindell, *AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption*, RFC 8452, April 2019.

[5] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schläffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology, 34(3), 2021.

[6] M. S. Turan, K. McKay, D. Chang, L. E. Bassham, J. Kang, N. D. Waller, J. M. Kelsey, D. Hong, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions*, NIST SP 800-232, August 2025.

[7] E. Barker, A. Roginsky, *Transitioning the Use of Cryptographic Algorithms and Key Lengths*, NIST SP 800-131A Rev. 2, March 2019.

[8] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of STOC '96, pp. 212–219, 1996.

[9] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, *Strengths and Weaknesses of Quantum Computing*, SIAM Journal on Computing, 26(5), pp. 1510–1523, 1997.

[10] C. Zalka, *Grover's quantum searching algorithm is optimal*, Physical Review A, 60(4), pp. 2746–2751, 1999.

[11] NIST, *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process*, NIST IR 8413, July 2022.

[12] A. Joux, *Authentication failures in NIST version of GCM*, NIST Comment, 2006.

[13] T. Iwata, K. Ohashi, K. Minematsu, *Breaking and Repairing GCM Security Proofs*, CRYPTO 2012, LNCS 7417, pp. 31–49.

[14] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, May 2010.

[15] E. Barker, J. Kelsey, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, NIST SP 800-90A Rev 1, June 2015.

[16] M. Bellare, C. Namprempre, *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*, ASIACRYPT 2000, LNCS 1976, pp. 531–545.

[17] P. Rogaway, T. Shrimpton, *A Provable-Security Treatment of the Key-Wrap Problem*, EUROCRYPT 2006, LNCS 4004, pp. 373–390.

[18] A. Rukhin, J. Soto, J. Nechvatal, et al., *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev 1a, April 2010.